WeatherSense: Web-Driven IoT & ML Platform for Real Time Weather Forecasting

By
Rashed Hossain
221-15-5679

Md Sadman Shakib
221-15-5401

FINAL YEAR DESIGN PROJECT REPORT

This Report Presented in Partial Fulfillment of the Requirements for the **Degree of Bachelor of Science in Computer Science and Engineering**

Supervised by

Mr. Md Mohammad Masum Bakaul
Senior Lecturer
Department of Computer Science and Engineering Daffodil International University

Co-Supervised by

Md. Abdullah-Al-Kafi, Lecturer
Department of Computer Science and Engineering Daffodil International University



DAFFODIL INTERNATIONAL UNIVERSITY
Dhaka, Bangladesh

May 10, 2026

APPROVAL

This Project titled "**WeatherSense: Web-Driven IoT & ML Platform for Real Time Weather Forecasting**," submitted by **Rashed Hossain** and **Md Sadman Shakib** to the Department of Computer Science and Engineering, Daffodil International University, has been accepted as satisfactory for the partial fulfillment of the requirements for the degree of B.Sc. in Computer Science and Engineering and approved as to its style and contents. The presentation has been held on **10-05-2026**.

BOARD OF EXAMINERS

Name
Board Chairman
Designation, Department of CSE, FSITDaffodil International University

---

Name
Internal Examiner 1
Designation, Department of CSE, FSITDaffodil International University

---

Name
Internal Examiner 2
Designation, Department of CSE, FSITDaffodil International University

---

Name
External Examiner
Designation, Department of CSE, FSITDaffodil International University

DECLARATION

We hereby declare that this project has been done by us under the supervision of **Mr. Md Mohammad Masum Bakaul**, **Senior Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this projectnor any part of this project has been submitted elsewhere for the award of any degree or diploma.

Supervised by:

Mr. Md Mohammad Masum Bakaul
Senior Lecturer
Department of Computer Science and EngineeringDaffodil International University

Co-Supervised by:

Md. Abdullah-Al-Kafi
Lecturer
Department of Computer Science and EngineeringDaffodil International University

Submitted by:

Rashed Hossain
221-15-5679
Department of Computer Science and EngineeringDaffodil International University

Md Sadman Shakib
221-15-5401
Department of Computer Science and EngineeringDaffodil University

©*Daffodil International University*

ACKNOWLEDGEMENTS

This work would not have been possible without the support and contributions of many individuals over the past two semesters. We are deeply grateful to everyone who has as- sisted us in one way or another.

First, we express our heartfelt thanks and gratefulness to the almighty for His divine blessing making it possible for us to complete the **Final Year Design Project(FYDP)** successfully.

We are grateful and wish our profound indebtedness to **Mr. Md Mohammad Masum Bakaul**, **Senior Lecturer**, Department of Computer Science and Engineering, Daffodil International University, Dhaka, Bangladesh. Deep knowledge and keen interest of our supervisor in the field of **Web Application** to carry out this project. His endless pa- tience, scholarly guidance, continual encouragement, constant and energetic supervision, constructive criticism, valuable advice, reading many inferior drafts, and correcting themat all stages have made it possible to complete this project.

We would like to express our heartfelt gratitude to the Head of the Department of Com- puter Science and Engineering, for his kind help in finishing our project and also to other faculty members and the staff of the Department of Computer Science and Engineering, Daffodil International University.

We would like to thank our entire course-mates at Daffodil International University, who took part in this discussion while completing the coursework.

Finally, we must acknowledge with due respect the constant support and patience of our parents.

ABSTRACT

Weather systems are vital to our lives, farming, travel and emergency management. But conventional weather monitoring and forecasting systems lack real-time data availability and localised accuracy. To overcome this limitation, this project introduces WeatherSense: Web-Driven IoT & ML Platform for Real-Time Weather Forecasting, a hybrid system integrating IoT, web development and machine learning for real-time weather monitoring and short-term prediction. The system includes an ESP32 microcontroller board interfaced with sensors to measure real-time temperature, humidity, air pressure and rainfall. The data is then sent to a server via HTTP and stored in a database. A web dashboard is created to display real-time weather data in a user-friendly manner. A machine learning algorithm is also developed to process historical and live data for weather forecasting. The use of hardware and software technologies allows for real-time monitoring and data presentation. The system enhances access to local weather data and offers real-time insights to aid decision-making. In summary, WeatherSense showcases the use of IoT and machine learning techniques to develop an affordable and scalable real-time weather monitoring system.

Table of Contents

Approval i

Declaration ii

Acknowledgements iii

Abstract iv

List of Figures vii

List of Tables viii

1 Introduction 1

1.1 Introduction 1
1.2 Motivation 2
1.3 Objectives 2
1.4 Methodology 3
1.5 Project Outcome 4
1.6 Organization of the Report 5

2 Background 7

2.1 Introduction 7
2.2 Literature Review 8
2.2.1 Similar Applications 12
2.3 Gap Analysis 14
2.4 Summary 15

3 Research Methodology 17

3.1 Methodology/Requirement Analysis & Design Specification 17
3.1.1 Overview 18
3.1.2 Proposed Methodology/ System Design 18
3.1.3 Functional and Nonfunctional Requirements 28
3.1.4 Data Flow Diagram 29
3.1.5 UI Design 31
3.2 Detailed Methodology and Design 35
3.3 Project Plan 36
3.4 Task Allocation 37
3.5 Summary 38

4 Implementation and Results 40
4.1 Environment Setup 40
4.2 Testing and Evaluation/Performance/ Comparative Analysis 41
4.3 Results and Discussion 43
4.4 Summary 48

5 Engineering Standards and Design Challenges 50
5.1 Compliance with the Standards 50
5.1.1 Software Standards 50
5.1.2 Hardware Standards 51
5.1.3 Communication Standards 51
5.2 Impact on Society, Environment and Sustainability 52
5.2.1 Impact on Life 52
5.2.2 Impact on Society & Environment 53
5.2.3 Ethical Aspects 53
5.2.4 Sustainability Plan 54
5.3 Project Management and Financial Analysis 55
5.4 Complex Engineering Problem 57
5.4.1 Complex Problem Solving 57
5.4.2 Engineering Activities 59
5.5 Summary 59

6 Conclusion 61
6.1 Summary 61
6.2 Limitation 61
6.3 Future Work 62

References 64

List of Figures

3.1 IoT architecture 20
3.2 Operational Workflow Diagram 21
3.3 System Activity Diagram 22
3.4 Use Case Diagram 23
3.5 System sequence diagram 24
3.6 Component Diagram 25
3.7 ESP32 Device State Diagram 26
3.8 Entities Relationship Diagram 27
3.9 Level 0 Data Flow Diagram 30
3.10 Level 1 Data Flow Diagram 31
3.11 User Dashboard Interface 32
3.12 Forecasting Interface 32
3.13 Sensor History Interface 33
3.14 Sensor Logs Interface 33
3.15 Extended Sensor Logs 34
3.16 Model Training Interface 35
4.1 Circuit Diagram 40
4.2 Training Feature from Sensor Data 45
4.3 Training Status Using Sensor Data 45
4.4 Forecast Parameters Interface 46
4.5 The Upload CSV Dataset interface 46
4.6 Training Status From External Data 47
4.7 Training Output From External Data 48

List of Tables

2.1 Summary of Literature Reviewed 9
2.2 Gap Analysis 14
3.1 Task Allocation 37
4.1 Comparative Analysis: Traditional Weather System vs API- Based Apps vs WeatherSense System 42
5.1 Proposed System Budget (Final Choice) 55
5.2 Alternative Budget (Greater Accuracy System) 56
5.3 Mapping with Complex Engineering Problem. 57
5.4 Mapping with knowledge Profile 58
5.5 Mapping with Complex Engineering Activities 59

Chapter 1

Introduction

This chapter gives a summary of the WeatherSense system, its purpose, and significance. It presents the idea of real-time weather monitoring and prediction with the help of IoT and machine learning. The background and motivation of developing such a system is also discussed in the chapter. Moreover, it also describes the major issues that the project will deal with and how the project is going to solve them. The primary goals and the range of the system are briefly outlined in order to have a clear picture of the work. Lastly, the anticipated results of the project are summarized to provide the basis of the subsequent chapters.

1.1 Introduction

Weather data is vital to our society, impacting agriculture, transportation, construction, disaster preparation and response, as well as individual activities. Weather forecasting plays a vital role in risk mitigation, planning and decision-making in both urban and rural settings. But traditional weather monitoring systems are typically based on fixed stations that offer macro-scale weather forecasts. Therefore, they may not provide accurate, timely and localised weather forecasts, particularly at the micro or local scale.

Recent developments in the Internet of Things (IoT) have provided opportunities for real-time monitoring of the environment. IoT allows for the deployment of connected sensors and devices that gather and share information from the real world. Meanwhile, web technologies can be used to display this data remotely and machine learning can be used to analyse historical and real-time data for predicting future outcomes.

This project, WeatherSense: Web-based IoT & Machine Learning Platform for Real-time Weather Prediction, aims to overcome the shortcomings of conventional weather monitoring systems by combining IoT, web programming and machine learning techniques. WeatherSense employs an ESP32 microcontroller to monitor weather data using sensors for temperature, humidity, atmospheric pressure, and rainfall. This information is then sent to a server and stored in a database for real-time processing and display via a web dashboard.

The main challenge of this project is the absence of cost-effective, scalable and real-time local weather monitoring systems that are accessible to the masses. WeatherSense seeks to offer an effective and viable solution by integrating sensing, real-time data transfer and prediction, to improve weather information access and support decision making

based on real-time and predicted weather conditions.

The motivation behind this project is the absence of cost-effective, scalable and real-time local weather monitoring systems that can be readily accessible to the general public. Through integration of hardware sensing, real-time data transfer and prediction, WeatherSense seeks to offer an effective and practical solution to improve access to accurate weather data and support decision-making based on current and forecasted weather conditions.

1.2 Motivation

This project is motivated by the increasing demand for precise, timely and local weather information that can be used to support everyday activities and life-threatening decisions. Weather reports in many areas, particularly in developing countries, are typically broad and not specific enough. This can be problematic for farmers, commuting and outdoor activities, where abrupt weather changes can be crucial. So the design of a system to deliver real-time local environmental data is an interesting computational and practical challenge.

This project is of interest from a computational standpoint as it has an opportunity to bring together a range of modern technologies including Internet of Things (IoT), web and machine learning. IoT allows for data gathering in real-time from physical sensors and web technologies create an interface that users can interact with to view the data. Further, machine learning brings the capability to learn from patterns and make short-term weather forecasts, increasing the system's intelligence and functionality.

Another important motivation is the increasing focus on developing smart data-driven systems that can work autonomously. Creating such a system improves skills as embedded systems, backend programming, and predictive analytics. It also offers practical experience with real data streams and data processing and visualisation techniques.

Finally, the problem is also valuable to solve because it enhances practical engineering skills and knowledge of the design of full-stack IoT systems. It also supports learning and development by creating a real-time system that showcases the use of computer science and engineering principles and concepts to solve environmental problems.

1.3 Objectives

This project was developed with several specific objectives that establish the system's requirements and focus:

- To develop an IoT hardware system based on ESP32 and environmental sensors to acquire weather data in real-time in terms of temperature, humidity, air pressure and rain status.
- To develop a web-based dashboard to present the real-time weather data in an interactive and user-friendly format.
- To implement a backend system for sensor data acquisition, processing, and storage in a proper database.
- To apply machine learning algorithms for processing historical and real-time data to produce short-term weather forecasts, and enhance forecasting accuracy.

These goals guarantee the successful implementation of a real-time, scalable and smart IoT system for weather monitoring and forecasting.

1.4 Methodology

The proposed project methodology is based on a comprehensive approach of hardware implementation, software programming and statistical analysis to implement real-time weather monitoring and forecasting. In the beginning, various weather data including temperature, humidity, air pressure and rain status are gathered by several sensors linked to an ESP32 microcontroller. The microcontroller serves as the main processing unit, collecting data from the sensors at a specified interval and preparing it for sending.

The data is then sent to a backend server via HTTP over a wireless network connection. At the server end, a backend system is coded using relevant technologies to collect, process and store the data in a database. The data is checked and formatted for consistency. This data is used as both current and historical data for analysis.

A web dashboard is also created and deployed to present a user-friendly and interactive real-time weather monitoring interface. The dashboard presents the real-time data in the form of charts, graphs, status bars, and other indicators, allowing users to remotely monitor the current environmental conditions. Moreover, the data is used to train a machine learning model that learns patterns and trends to make short-term forecasts.

The entire process is a loop, allowing for real-time data gathering, processing, display and forecasting. This approach allows effective coordination between hardware and software elements, scalability, accuracy and user-friendliness.

Requirement Analysis:

The system requirements are determined in this stage, according to the real world needs. The key specifications are the real-time gathering of weather information, correct sensor measurements, wireless connectivity, data repository and easy visualization. Functional and non-functional requirements are examined to provide adequate system design and

functionality.

System Architecture and System Design:

The system architecture is then created after the requirement analysis to establish the interaction of various components amongst themselves. This involves sensor-ESP32 connection, flow of data to the server at the back-end, database format, and communication between the back-end and front-end. The design will provide seamless integration and real-time functionality of the system.

Data in the Environment Modeling and Implementation of Features:

Sensors attached to ESP32 are used to gather environmental data like temperature, humidity, air pressure, and rainfall at this stage. This is done by modeling and structuring the data properly and then storing in the database. The main functions are real-time data collection, data processing, and prediction based on machine learning and are applied in this step.

Graphical User Interface:

An online dashboard is created to display the current and past weather data. The interface displays information in the form of graphs, charts and status indicators to ensure it is easy to understand. The UI aims to be user-friendly, responsive, and easy to access by making it more accessible and user-friendly.

Combination of Testing and Feedback:

The last stage is the integration testing whereby appropriate communication among the various parts of the system is tested. This involves testing the accuracy of the sensor, data transmission, processing at the backend, and display on the frontend. Testing provides feedback to enhance the performance, stability, and reliability of the system.

1.5 Project Outcome

The main deliverable of this project is the successful implementation of an integrated IoT weather monitoring and prediction system that offers real-time weather information via a web-based system. This will allow users to track and observe weather data including temperature, humidity, air pressure, and rainfall from anywhere with an internet-connected device.

The other key outcome is the development of a forecasting system based on machine learning. The system will use historical and current data for short-term weather predictions, providing added value beyond data monitoring. This feature can help users plan their activities, farming, and outdoor events more effectively.

Furthermore, the project will deliver an affordable and expandable system that can be installed in several places for localised weather monitoring. The system's modularity enables scalability for future needs, such as incorporating more sensors or features.

Academically and practically, this project's product showcases the successful application of IoT, web and machine learning technologies. This project boosts expertise in embedded systems, server-side programming, data manipulation and machine learning, and also helps address real-world issues in environmental monitoring.

1.6 Organization of the Report

The report is divided into a number of chapters that cover different aspects of the project to improve readability, flow and comprehension. This structured chapter-based presentation is designed to provide detailed information on the design and implementation of WeatherSense: Web-Driven IoT & ML Platform for Real-Time Weather Forecasting.

Chapter 1: Introduction - This chapter outlines the project, including the background of the weather monitoring system, the need for the work, the well-described goals, methods and outcomes. This chapter introduces the report by describing the problem and the solution.

Chapter 2: Background - This chapter explains the basic concepts and technologies used. It covers the fundamental concepts of Internet of Things (IoT), sensing environmental data, web portal for data visualization, and simple machine learning algorithms for predicting weather. This chapter lays the foundation for the project.

Chapter 3: Requirement Analysis & Design Specification - This chapter describes the analysis of system requirements (functional and non-functional). It also discusses the system architecture, design diagrams, data flow, and hardware and software specifications of the system.

Chapter 4: Development and Results - This chapter describes the implementation of the system, including the hardware configuration (ESP32 and sensors), the backend system, database connection, and the web dashboard. It also reports the findings of real-time data monitoring and results of the prediction model.

Chapter 5: Engineering Standards and Design Challenges – This chapter discusses the engineering standards, practices, and design aspects of the project. It also explains the challenges, including sensor accuracy, network stability and data management, and the techniques used to address these issues.

Chapter 6: Conclusion - This chapter concludes the project by summarising the work, achievements and contributions. It also outlines the shortcomings of the system and suggests future work, such as improving the prediction model accuracy,

incorporating additional sensors, or creating a mobile app.

In summary, this approach ensures that the report describes the project in a clear, comprehensive and professional way, facilitating the understanding of the different phases of development and evaluation.

Chapter 2

Background

The chapter has the background knowledge that is needed to grasp the WeatherSense system. It describes the fundamentals of IoT, embedded systems, and machine learning that are applied in this project. Another issue touched upon in the chapter is the functioning of weather monitoring systems in general and the necessity of real-time data collection as the key to a proper analysis of the environment. It also briefly examines the current technologies and systems to determine their weaknesses and areas of enhancement that they require. In general, this chapter forms the basis of the explanation of how the proposed system of WeatherSense would be designed.

2.1 Introduction

This section will give the basic background knowledge needed to comprehend the concepts, technologies, and processes involved in developing WeatherSense: Web-Driven IoT and ML Platform to Real-Time Weather Forecasting. To effectively design, implement and evaluate the proposed system, it is necessary to learn the underlying technologies like IoT, machine learning, and web-based data visualization systems.

Weather monitoring systems are developed to acquire and analyze environmental information like temperature, humidity, air pressure and rainfall. Conventional weather forecasting systems are usually based on large-scale weather stations that deliver generalised forecasts of weather across large geographic areas. Nonetheless, none of these systems have real-time, localized forecasting capabilities, which are necessary in modern applications like smart agriculture, transportation, and disaster management [7], [15].

With the advent of the Internet of Things (IoT) the environmental monitoring system has greatly been enhanced through the ability to collect real time data using sensors and microcontrollers. Continuous data collection in the physical environment and transmission to the servers or cloud platforms are part of IoT-based systems [1], [8], [11]. They can effectively track local weather conditions and they have found wide usage in smart weather station applications [9], [12].

Moreover, web-based technologies and mobile applications are also important in visualizing real-time weather information. Interactive dashboards enable the user to view live environmental conditions and remotely retrieve forecasting results, which enhances usability and accessibility of weather systems [1], [14]. Cloud-based architectures can also increase the scalability of an IoT system and provide the

opportunity to maintain centralized data management [6].

Moreover, machine learning methods are commonly applied to studying the gathered weather data and produce short-term forecasts. Machine learning models, including regression, neural networks, and hybrid models, can enhance the accuracy of the forecasts made by utilizing historical and real-time sensor data [2], [5], [17]. Recent studies also emphasize the combination of edge and fog computing and ML models to minimize the latency and enhance the real-time decision-making [13], [17].

The combination of IoT, machine learning, and cloud/web technologies is the fundamental base of the contemporary intelligent weather forecasting systems. It is the main goal of the proposed WeatherSense system [1], [6], [14] that, with the use of these combined technologies, smart systems are developed, which can be used in real-time monitoring, prediction and visualization, which is the primary objective of the proposed WeatherSense system [1], [6], [14].

2.2 Literature Review

The In this section, a review of all the existing studies on the IoT-based weather monitoring systems, machine learning-based predictive models, and hybrid intelligent architectures will be presented. Integration of IoT and machine learning has emerged as a significant research focus towards real-time monitoring and prediction systems of the environment. A number of studies have been dedicated to sensor-based weather data collection using IoT devices and microcontrollers like ESP32 that allow real-time acquisition of temperature, humidity, rainfall, and atmospheric pressure data [1], [8], [11], [12]. These systems prove that IoT technology can be very effective in monitoring the continuous environmental data and in low cost implementation in the real world.

Along with hardware-based systems, numerous researchers have used machine learning algorithms to enhance the accuracy of weather forecasting. To illustrate, collaborative and multi-location prediction models have performed well in comparison to traditional single-source forecasting methods [2]. Likewise, hybrid strategies involving IoT, artificial intelligence, big data, and cloud computing have been suggested to enhance the reliability of predictions and their scalability [6]. The recent research also emphasizes the implementation of the advanced ML methods like anomaly detection, fog computing, and edge-based intelligence to improve the accuracy of forecasting and the robustness of the system [13], [17].

To visualize real time weather data, several mobile and web-based applications have been developed to make forecasting systems user friendly and accessible [1], [14]. With these applications, users can view live environmental conditions, and get

predictive insights via dashboards and mobile interfaces. Systematic reviews also confirm that the incorporation of IoT sensors into systems can greatly enhance the real-time weather monitoring systems, but issues related to scalability and prediction precision still persist [3], [15].

Additionally, scholarly studies conducted on smart agriculture prove the significance of weather prediction in practical activities. Crop monitoring, irrigation planning and environmental decision-making are also widely done using weather prediction models [10], [16], [19]. The agricultural forecasting systems based on machine learning indicate that effective weather forecasting can greatly enhance productivity and resource management [20].

Other researchers have been interested in lightweight and scalable IoT-based weather stations with a focus on low-cost deployment and efficient data transmission [8], [9]. It has also been suggested that adaptive real-time forecasting systems can be used to enhance the accuracy of prediction using dynamic environmental data [7]. Moreover, onboard and embedded prediction systems prove that edge based computation is feasible in real time weather prediction [12].

Even though there have been prolific developments, the current systems still have a number of limitations. Most IoT-based solutions are able to collect data but not predict it with advanced analytics, and many ML-based models are not able to predict based on real-time sensor data but require more detailed data analysis. Furthermore, most of the existing research is limited to integration between IoT devices and machine learning models, databases, and web dashboards. Thus, there is an obvious research gap in developing a fully integrated, real-time, and lightweight weather monitoring and forecasting system that has integrated IoT, machine learning and web technologies into a unified architecture, which is the focus of the proposed WeatherSense system.

Table 2.1: Summary of Literature Reviewed.

Author (s)	**Year**	**Title**	**Methodology**	**Key Findings**
J. J. A.-M. Sayor et al. [1]	2025	IoT-driven real-time weather measurement and forecasting mobile application with machine learning integration	IoT + ML integration	Mobile (ML) real-time weather forecasting

T. P. Fowdur et al. [2]	2022	Real-time collaborative machine learning based weather forecasting system	Multi-location ML model	Improved forecasting using multiple predictors
S. K. Das and P. Nayak [4]	2024	AI for Financial Forecasting	AI-based IoT system	Improved forecasting of agriculture
D. Bhattacharyya and I. Banerjee [5]	2025	Synergizing IoT and machine learning in hybrid models	Hybrid ML architecture	Better prediction efficiency and accuracy
M. Kumari et al. [6]	2021	Smart agriculture weather forecasting using IoT and cloud	IoT + Cloud + ML	Scalability and decision support are improved
T. P. Fowdur et al. [7]	2018	Adaptive real-time weather forecasting system	Adaptive algorithm model	Real-time environmental monitoring
Abdullah Albuali et al. [8]	2023	Lightweight IoT-based smart weather measurement system	Embedded IoT system	Low-cost scalable monitoring system
Bella et al. [9]	2023	Sustainable IoT-based smart weather station	IoT sensor network	Real-time environmental monitoring
M. D. S. Sharafat et al. [10]	2025	IoT-enabled AI crop prediction system	AI + IoT model	Prediction weather-based agricultural
R. Kumar et al. [20]	2025	Smart farming & weather forecasting with ML	ML-based system	Increased agricultural decision making
B. İşler et al. [17]	2025	Fog-enabled ML approaches for weather prediction	Fog computing + ML	Reduced latency and improved accuracy

P. Karvelis et al. [12]	2020	Lightweight onboard weather prediction system	Embedded ML model	Edge-based efficient forecasting
M. Sakib et al. [18]	2024	Rainfall prediction and real-time weather monitoring in Bangladesh	ML comparative analysis	Better forecasting of rainfall accuracy
Avines P. Selvam and N. Saud [15]	2024	Systematic review of smart weather monitoring systems	Review study	Determined system constraints and tendencies

2.2.1 Similar Applications

Multiple IoT-based weather stations and mobile apps have been created to offer real-time weather information and prediction services. These systems are more or less similar to the proposed WeatherSense: Web-Driven IoT and ML Platform, Real-Time Weather Forecasting, in particular, sensor-based data collection, cloud integration, and visualization of the weather. Every system, however, is different in terms of architecture, scalability and intelligence level.

ESP32-Based IoT Weather Station:

The ESP32-based IoT Weather Station systems that use sensors like DHT11, DHT22, and BMP280 to detect the environmental conditions such as temperature, humidity, and air pressure are one of the widely used methods. These systems transmit information to web dashboards via Wi-Fi modules and show the outcomes in real time in graphs and charts. Flask, Node.js or Arduino IoT Cloud are used many times as visualization and data processing. These systems have a good real time tracking record, but tend to be limited in having advanced machine learning prediction capabilities and scalability to larger scale deployment.

Cloud-Integrated IoT Weather Platforms (ThingSpeak, Firebase, ThingsBoard):

Cloud-integrated IoT weather monitoring platforms are another category, which involves transmitting sensor data to cloud services like ThingSpeak, Firebase, or ThingsBoard. Remote access, data logging, and analytics are possible through these systems. There are also more sophisticated ones that have machine learning weather prediction algorithms based on past data. Nevertheless, most of such systems rely on third-party systems and this restricts customization and complete control over data processing and visualization.

Hyper-Local IoT Environmental Monitoring Systems:

Hyper-local environmental monitoring systems are also available, which concentrates on small-scale weather monitoring with distributed sensor networks of IoTs. These systems are very accurate to localized weather conditions and in some cases, they have anomaly detection and forecasting with machine learning models. As much as these systems are technically advanced, they are usually complex, expensive and not easily implementable on small scale academic or personal projects.

Weather Underground:

Weather Underground is a famous weather service, which gives real time weather conditions and forecasts, using a vast network of weather stations. It provides in-depth weather maps, radar, and prediction services. Nevertheless, it relies on centralized data sources but not user-controllable IoT sensors.

AccuWeather:

AccuWeather is a popular weather forecasting application that gives global weather forecast, weather alerts and radar images. It is based on sophisticated meteorological models without the support of local sensor integration relying on IoT.

The Weather Channel:

The Weather Channel offers real time weather report, extreme weather, and weather forecasting. It is very reliable and is primarily based on centralized weather data collection systems.

WeatherBug:

WeatherBug is a real-time weather tracking and lightning detector. It is based on a massive sensor network and lacks the ability to customize the IoT to the needs of the user.

Windy:

Windy is an impressive weather visualization tool and it gives detailed wind, temperature and storm tracking maps. It is very sophisticated in visualization and does not allow integration of custom hardware.

RainViewer:

RainViewer is devoted to real-time radar of precipitation and storms tracking. It offers radar visualization of high quality but is restricted to visualization of weather data only.

NOAA Weather Radar:

NOAA weather radar offers official weather forecasting and radar-based weather forecasting. It is very precise yet cannot be tailored to personal IoT-based systems.

MyRadar:

MyRadar is an app on the phone that delivers real-time radar images and alerts. It is easy to use and relies on centralized data sources.

Yahoo Weather:

Yahoo Weather is a clean and easy weather forecasting site that has simple real-time forecasts. It is not hard to employ and does not have high-tech IoT or predictive customization.

Google Weather:

Google Weather is a search engine that offers instant weather forecasts. It is easy to use yet lacks in-depth analysis of the environment and customization.

Moreover, there are some mobile and web-based software that offers weather forecasting services to the users. The most popular ones are Weather Underground, AccuWeather, The Weather Channel, WeatherBug, Windy, RainViewer, NOAA Weather Radar, MyRadar, Yahoo Weather, and Google Weather. These applications have real-time updates, radar images and forecasting. Nevertheless, their main use is based on centralized sources of meteorological data instead of user controlled sensor networks of IoT, thus they are not capable of offering genuinely localized environmental monitoring.

2.3 Gap Analysis

The gap analysis reveals the limitations of the current IoT weather monitoring systems, cloud-based weather applications, and mobile weather applications. Based on the analysis of the similar literature available, it is noted that the majority of the available systems offer either real-time weather visualization or prediction separately, and do not offer a complete solution that integrates both IoT-based data collection with web-based visualization and machine learning-based prediction in a single system. Most systems also rely of third-party weather APIs, rather than attempting to gather localized environmental data using sensors, further degrading the accuracy of micro-level forecasting. Also, not all systems have user-friendly dashboards, customization, and real-time predictive analytics.

The other significant drawback is that the majority of solutions available cannot offer a single system in which raw sensor data is being processed and fed into a machine-learning-based short-term forecasting system. In most instances, there is no in-depth analysis or intelligence of data presented. Scalability and system independence are also a concern with a number of platforms being highly dependent on external cloud services, thus restricting flexibility and control over data processing.

WeatherSense is a proposed system that will address these restrictions by bringing IoT sensors, a web-based dashboard, and machine learning together into a single system. It is concerned with the real-time data capturing, localized weather surveillance, and predictive data analysis, which are not entirely supported by most of the existing systems.

Table 2.2: Gap Analysis

Features	IoT Weather Station Systems	ThingSpeak / Cloud Platforms	Mobile Weather Apps	Existing Web Dashboards	Weather APIs Based Systems	Proposed WeatherSense System
Real-time sensor-based data collection	Yes	Yes	No	Yes	No	Yes

Localized weather monitoring	Limited	Limited	No	Limited	Limited	Yes
Machine learning-based forecasting	No	Limited	No	No	Limited	Yes
Web-based dashboard visualization	Yes	Yes	No	Yes	Yes	Yes
Mobile app accessibility	No	Yes	Yes	No	Yes	Yes (future scope)
Dependency on external weather APIs	No	Yes	Yes	Yes	Yes	No
Custom hardware integration (ESP32/sensors)	Yes	Limited	No	Yes	No	Yes
Historical data storage & analysis	Limited	Yes	Yes	Limited	Yes	Yes
User-controlled system (full ownership)	Yes	No	No	No	No	Yes
Predictive analytics with real-time data fusion	No	Limited	No	No	Limited	Yes

2.4 Summary

This chapter will give the background knowledge that is needed to comprehend the development of the WeatherSense: Web-Driven IoT & ML Platform to Real-Time Weather Forecasting system. It starts by describing the significance of weather monitoring in our contemporary world and particularly in fields like agriculture, transportation, disaster control, and day-to-day planning where timely and accurate information concerning the environment is very crucial.

The discussion also shows the shortcomings of the conventional weather forecasting systems that are normally centralized and in many cases fail to deliver highly localized or real time information. This drawback leaves a loophole, which can be successfully filled with the help of modern technologies, including the Internet of Things (IoT). With the IoT, it is possible to collect data on the physical environment constantly with the help of sensors and microcontrollers such as ESP32 and conduct more accurate and place-specific weather observations.

Moreover, in the chapter, the author presents the importance of web technologies in providing real time information by way of interactive dashboards, which will help users to comprehend and interpret weather conditions easily. The relevance of data visualization as an important element in enhancing usability and decision-making are highlighted.

In addition, the chapter briefly explains how machine learning has improved weather forecasting since it uses past and real-time data to make short-term predictions. This brings an intelligent touch to the system and not just mere display of data but predictive analytics.

In all, this chapter lays the basic principles and technologies that should be used in the project. It establishes a clear picture of the collaboration between IoT, web development, and machine learning to develop the foundation of the proposed WeatherSense system that will provide a real-time, scalable, and intelligent system of weather monitoring.

Chapter 3

Research Methodology

The chapter explains the general approach used in the development of WeatherSense system. It provides the systematic nature of the strategy to be used to design, implement and assess the IoT-based weather monitoring and prediction system. The chapter addresses the requirements analysis, system design, data model, implementation strategy and testing process. It also brings out the integration of various elements like hardware, software and machine learning to make up the final system. All in all, this chapter clearly gives a road map of how the research and development process was conducted.

3.1 Requirement Analysis & Design Specification

The Requirement Analysis and Design Specification step establishes the functional requirements, system behavior and structural design of the WeatherSense: Web-Driven IoT and ML Platform for Real-Time Weather Forecasting system. This is necessary to make sure that the system is designed in a systematic way and achieves all the desired goals effectively.

The main specifications of the system are to gather real-time information about the environment, including temperature, humidity, air pressure, and rainfall status by means of IoT sensors to an ESP32 microcontroller. The system should also be able to guarantee the constant data acquisition and the reliability of the data transmission to the backend server via wireless communication. The backend system must take the incoming data, sanction it and store it in the structured database to be used further.

The other essential need is that a web-based dashboard should be developed where users are able to view real-time weather conditions in an interactive and user-friendly manner. The dashboard must be capable of graphical presentation of information like charts and live updates so that it becomes easy to interpret the environmental conditions.

On the non-functional aspect, the system must be scalable, reliable, and in a position to accommodate continuous data flow without big delays. It must also guarantee the accuracy of data, responsiveness of the system and minimum data downtime.

The system is specified in terms of design, and is broken down into four key parts: IoT Hardware layer, communication layer, backend server and frontend web interface. The data collection process is carried out by the IoT layer, data transmission is carried out by the communication layer, processing and storage is

carried out by the backend, and the visualization is provided by the frontend. Further it also includes a machine learning module that can analyze past data and provide short term weather forecasts, improving the system intelligence.

3.1.1 Overview

The phase of the WeatherSense: Web-Driven IoT & ML Platform For Real-Time Weather Forecasting project under consideration, the requirement analysis and design specification gives a clear vision of how the system is going to be designed and what functionality it will provide. This stage serves as an intermediary between the preliminary problem definition and the actual implementation of the system by determining the fundamental requirements and outlining the overall system design concept.

It is developed to meet the necessity of using real-time and local weather monitoring because it combines IoT hardware, web technologies, with machine learning methods into one platform. The IoT element has the task of gathering environmental information with the help of sensors attached to ESP32 microcontroller. This information is subsequently sent to a back-end server where it is processed and stored.

The online aspect of the system makes users find it easy to access and view real-time weather data on a dashboard-like interactive dashboard. This enhances usability and lets us monitor, in real time, the environmental conditions anywhere.

Moreover, the system design will incorporate a machine learning component that will handle both past and real-time data to create weather predictions of the short term. This increases the ability of the system not just to do straightforward monitoring but also to be able to make smart predictions.

On the whole, the general description of this stage shows the entire structure of the system and the way various elements are interrelated to each other to produce real-time data collection, processing, visualization, and prediction in one and efficient way.

3.1.2 System Design

The system design of WeatherSense: Web-Driven IoT and ML Platform to Real-Time Weather Forecasting explains the general structure of the system and the interaction between various system elements. The design is designed in such a way that it has efficient data flow between hardware sensing units and the end user interface, yet has real time processing and prediction capability.

The system is partitioned into four major layers which are the IoT sensing layer, the communication layer, the backend processing layer and the frontend visualization layer. All the layers are performed to ensure a smooth running of the system.

The IoT sensing layer is the ESP32 microcontroller inter-linked with environmental sensors which include temperature sensor, humidity sensor, air pressure sensor and rain sensor. This layer has the role of gathering real time weather data in the surrounding at specific intervals.

The communication layer facilitates an exchange of data between the IoT device and the

back-end server. The ESP32 transmits the obtained data via a wireless channel (HTTP requests via Wi-Fi) which guarantees the constant and real-time data flow.

The backend processing layer will help to receive, validate, process and store the sensor data received into a structured database. This layer is also in charge of information retrieval to be visualized and the machine learning module, which processes historical and real-time data to produce short-term weather predictions.

The frontend visualization interface is a web-based dashboard showing the real-time weather information in an interactive format. It operates with charts, graphs, and live indicators to show the environmental conditions to the user in a clear manner. This is a layer that will make access to weather information effortless through all devices that have an internet connection.

A. IoT System Architecture:

Figure 3.1: IoT System Architecture.The WeatherSense system takes the form of five unique functional layers that enable the process of converting raw environmental data into actionable insights. It starts at the Hardware Layer, where an ESP32 Microcontroller communicates with four main sensors to gather temperature, rain, pressure and humidity data.

After capturing these environmental parameters, the ESP32 packages them into "Sensor payloads" and sends them via the Network / Communication Layer using WiFi/ HTTP protocols. The Backend/Application Layer accepts these payloads in the form of HTTP requests and runs on a Backend Server powered by Flask. This server is the central API, its the logic of the system, it regulates the flow of data, between storage and processing units.

The smarts of the system are in the Data/ML Layer where raw data is stored in a Database and then taken to a Machine Learning Prediction Module. This module examines both historical and real-time data to conduct inference, which would result in weather forecasts and possible notification.

Lastly, these outcomes (such as real-time updates and predictive alerts) are displayed on a Web Dashboard/User Interface in the Presentation Layer. This stratified solution guarantees that the sensor-based data, which comes in at high frequencies, is efficiently processed and converted into a friendly format to be used in environmental monitoring..

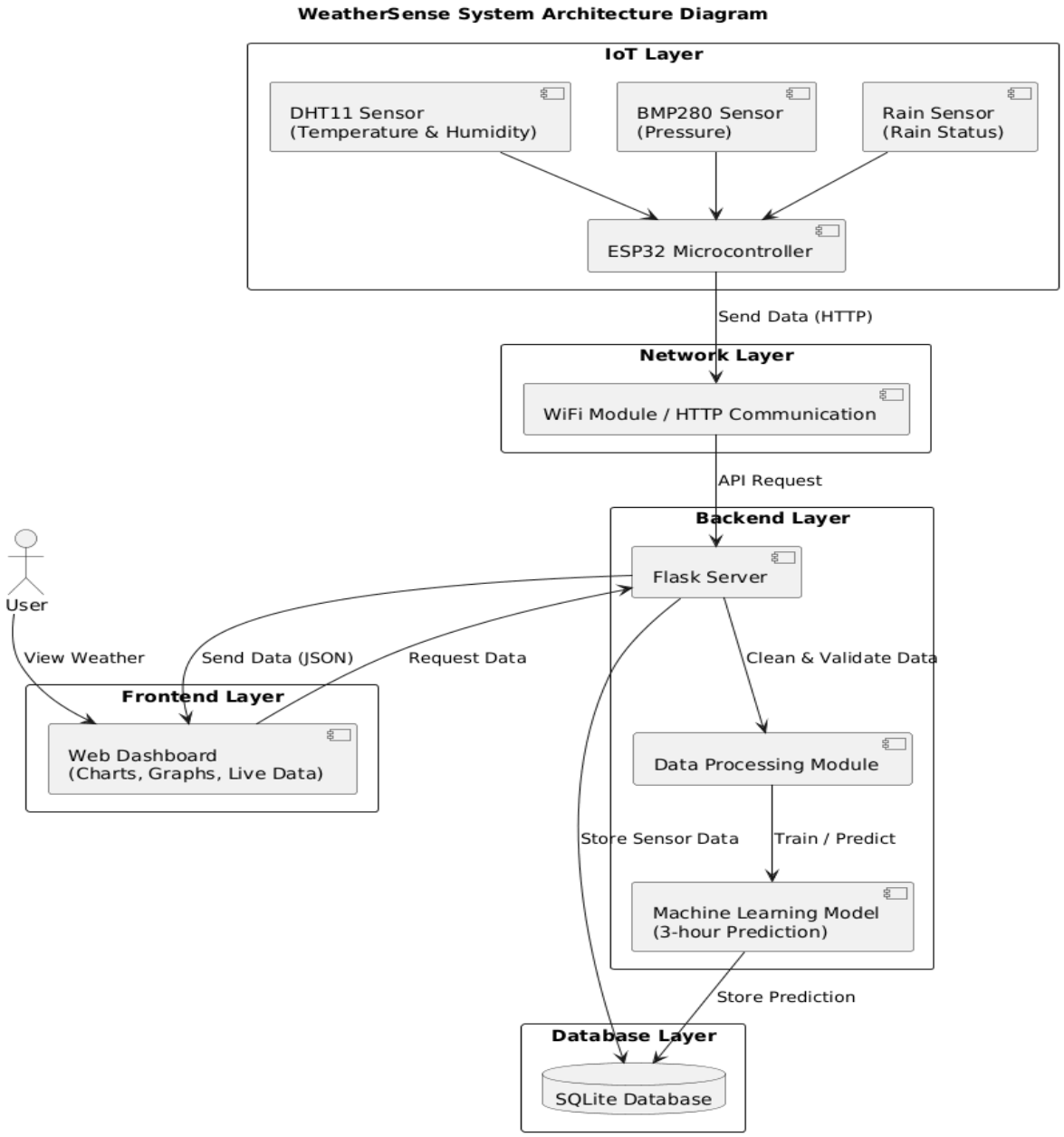


Figure 3.1: System architecture

B. Operational Workflow Diagram:

The Operational Workflow (Figure 3.2) describes the end-to-end logical sequence of data through in this system, as it passes between raw physical input and also actionable user intelligence. The cycle begins with the gathering of information with sensors and the environmental variables are converted into digital form by the hardware. This data is automatically transferred to the server and stored in the database to have a complete record of history.

The web dashboard is updated in near real-time to ensure high system responsiveness, so that users can have instant access to current conditions. After the data persistence

phase, the system calls upon the ML model to make a prediction based on the new telemetry to predict future weather changes. This is finished by an automated check to send notifications in case of a requirement without the process being restarted again once more to capture the data..

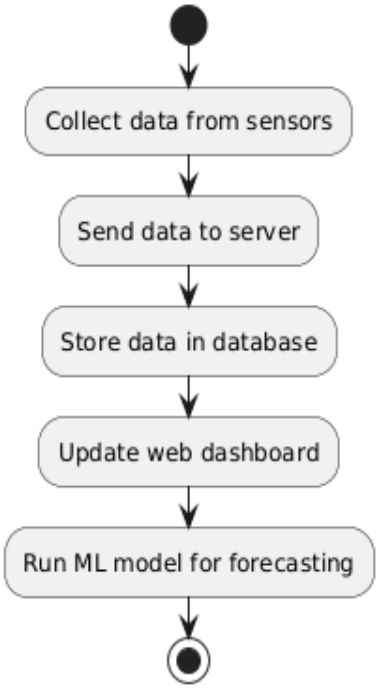


Figure 3.2: Operational Workflow Diagram

C. System Activity Diagram

Figure 3.3: System Activity Diagram, in detail. The process starts with the Power on and the initialisation of sensors to ensure that all the hardware is calibrated and prepared. The system then goes into an endless loop in which it reads the raw data (Temperature, Humidity and Rainfall). Importantly, the workflow version has a Validate and filter noise step that can be used as a data-cleaning step to guarantee the correctness of the data prior to transmission. After the data packet has been cleaned, it is sent to the server where a security check is performed; the server verifies the request and logs the data at the same time.

After the successful transfer of data to the database, as well as the refresh of the real-time dashboard, the system will enter an important decision node: "Data threshold reached to forecast?

- No: The system will forego the forecast and go back to the data collection stage to conserve the computational power in case there is not sufficient new information to create a significant update.

- When Yes: The logic calls the ML forecast model, continues to make a forecast report.

This decision logic logic is what makes the WeatherSense system efficient and responsive to emergency environmental changes.

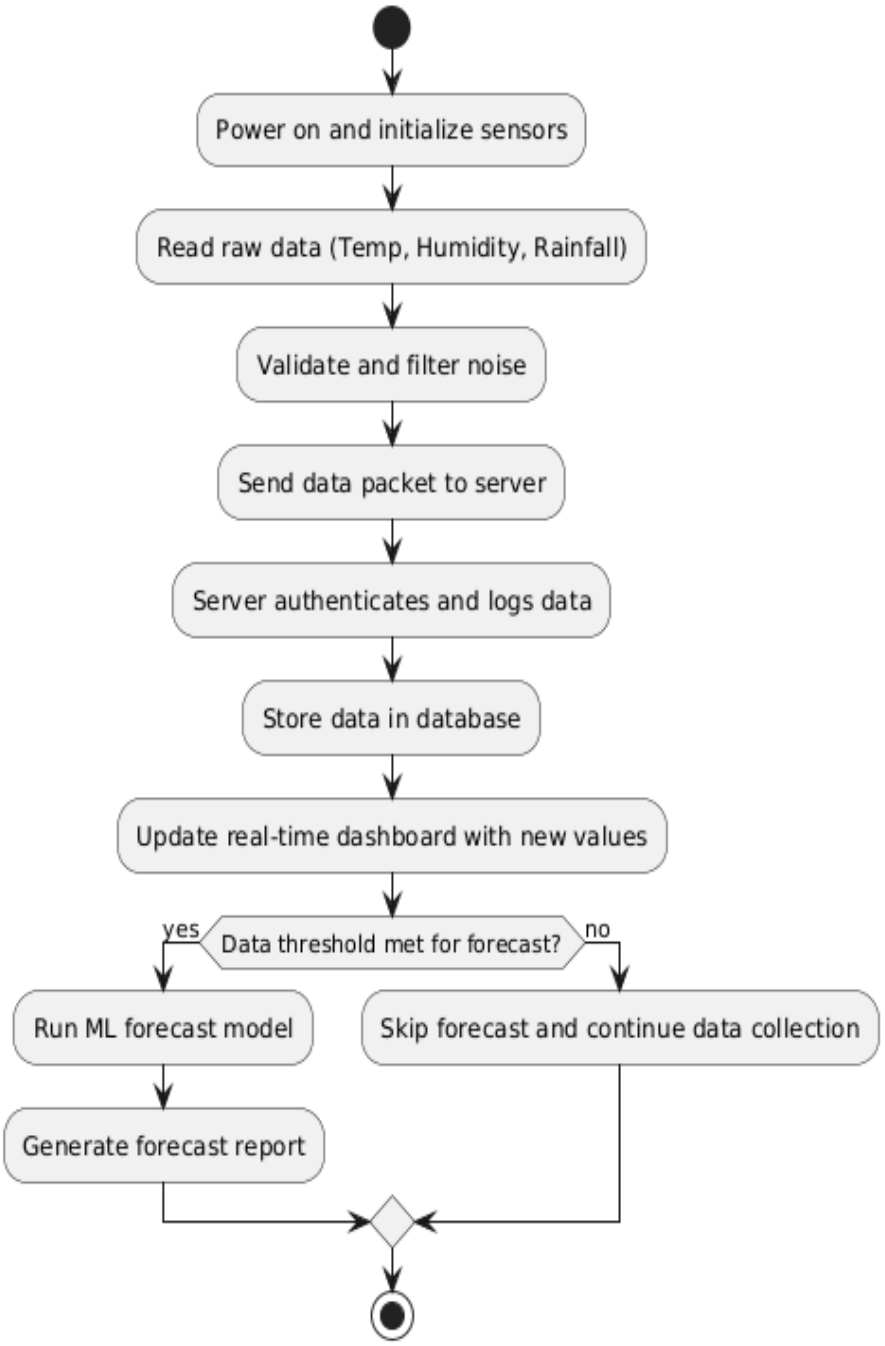


Figure 3.3: System Activity Diagram

D. Use Case Diagram

The Use Case Diagram (Figure 3.4) describes the main actor, the User, and delineates their particular permissions and interactions within the system boundary. The User role revolves around the use and interpretation of environmental data using various fundamental functional pathways. These are capability to do the following: Login/Authentication, which is the ability to access the platform safely. After authentication, the User will be able to View Real-Time and Historical Data giving them a current view of the current weather conditions as well as a historical perspective of the

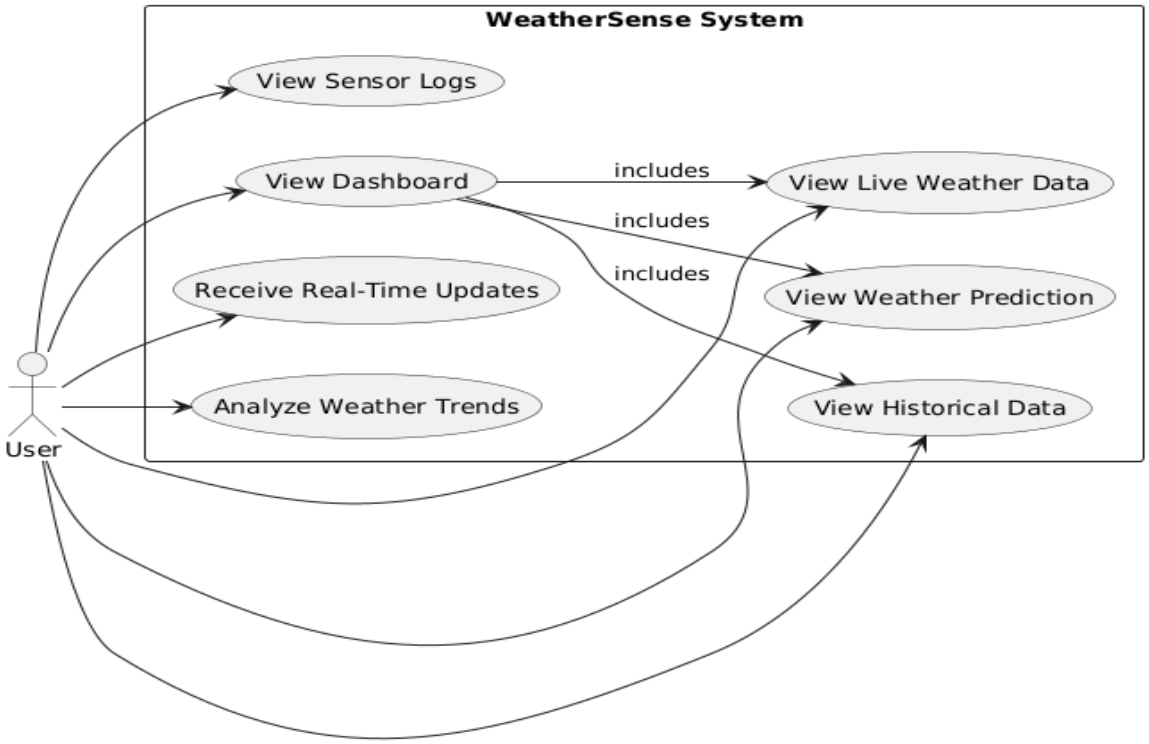


Figure 3.4: Use Case Diagram

past weather patterns. Moreover, the User may Get Weather Forecasts which are based on the predictive weather models used by the system and Export Weather Data to be used by other parties or further report on the subject. These use cases delineate the general functional flow of an observer, so that all the essential weather data and tools that facilitate sharing of data and information are readily accessible to the end-user without creating a complex and convoluted application experience.

E. System Sequence Diagrams

System sequence diagram is presented in Figure 3.5. The setup starts with the ESP32 unit which sends POST request with sensor measurements (temperature, humidity, pressure and rain) to the Backend Server. The server on receiving them does an INSERT operation to save these readings to the Database and in response sends him or her a confirmation ID. The server then forwards a POST /infer request to the Machine

Learning Model in order to produce insights. The model interprets the data and sends its output and analysis to the server. After this processing process the server then sends the output to the Web UI which then shows the real-time weather output to the User.

The second step of the diagram illustrates a request by a user. The Web UI sends a GET request to the Backend Server when a User requests particular weather data (via ranges or filters) via the Web UI. The Database provides the server with the corresponding historical readings and predictions. When the database gives out the result set, the server responds with 200 OK and the requested weather data to the Web UI, which ultimately displays the information to the user. This two-way flow illustrates the automated data ingestion as well as the on-demand data retrieval features of the architecture.

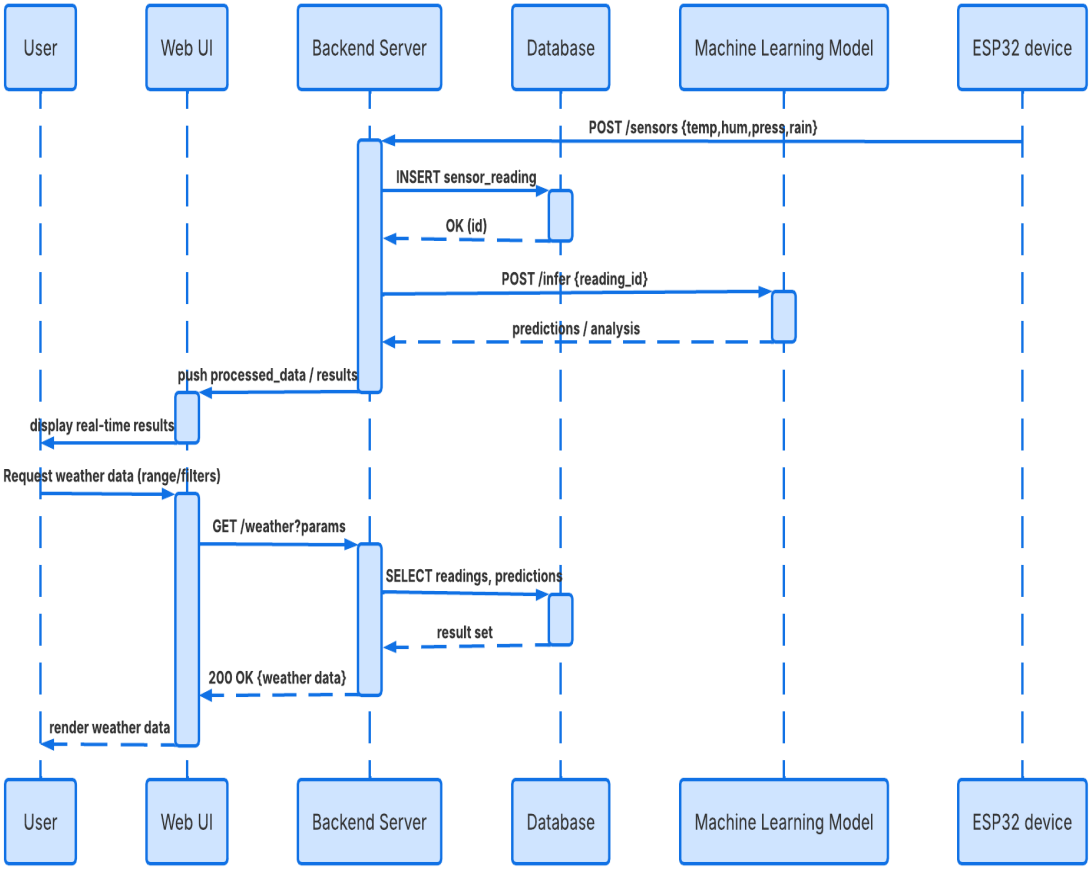


Figure 3.5: System sequence diagram

F. Component Diagram

Figure 3.6: Component Diagram. The design node is the Backend API (Flask) that acts as the main coordinator of all data transactions. Here four primary communication channels supported by this centralized hub:

- **Data Ingestion:** ESP32 Sensor Module serves as a source of raw data, where the

module sends the raw information about the environmental to the Backend API, through POST requests.

- **Persistent Storage:** The API communicates with the Database both ways, placing commands to store readings and get a response in the form of an acknowledgement or a unique ID and also requesting historical data on behalf of the user.

- **Intelligence Integration:** The Backend API interacts with the Machine Learning Engine, providing data to infer and returning weather forecasts to forecast.

- **User Interaction:** The Web Dashboard UI interacts with the API to request the weather information and the API provides the result in the form of processed information to be displayed.

This modular architecture makes sure that the data collection, storage, analysis, and presentation modules are not coupled and so each system component can be extended or refined separately.

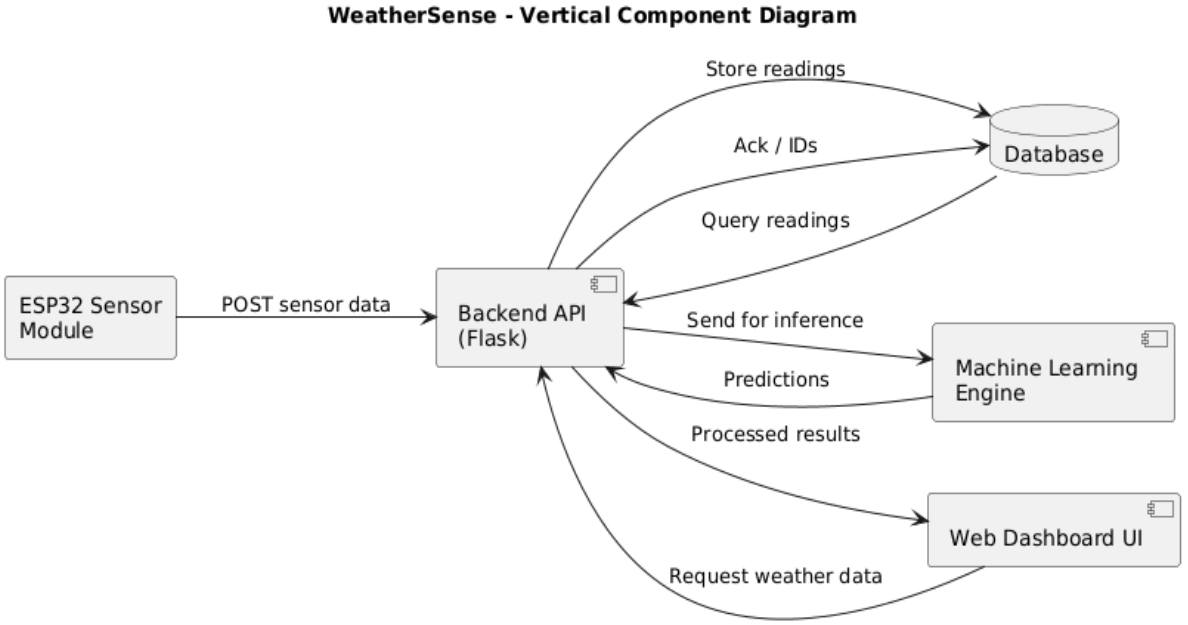


Figure 3.6: Component Diagram

G. ESP32 Device State Diagram

Figure 3.7: ESP32 Device State Diagram. The device is a state machine that goes through a sequence of logical states in order to provide effective power consumption and data transmission reliability. When the device begins a booting system, it gets into an Idle state, which consumes less power until a certain timer or event causes the next stage. Upon activation, the device switches to Reading Sensors, in which the device reads environmental samples. Once the samples has been captured successfully and the device

goes into the Processing Data state, where the raw signals is formatted into a readable structure.

After preparation of data, the device goes to Sending Data where it encodes the information and transmits the telemetry to the backend server. The last critical state is Waiting for Response, a decision node that decides the course of action to take next depending on the feedback of the network:

- **Acknowledgment (ACK) Received:** When the server acknowledges that it has received the data, the device goes back to the Idle state until the next planned reading.

- **Timeout/Immediate Cycle:** In case of a time-out or the system is set to continuous monitoring, the device goes directly to Reading Sensors state to capture again.

This governmental solution guarantees that the ESP32 is not only receptive to network environments but also allows a well-organized stream of weather data.

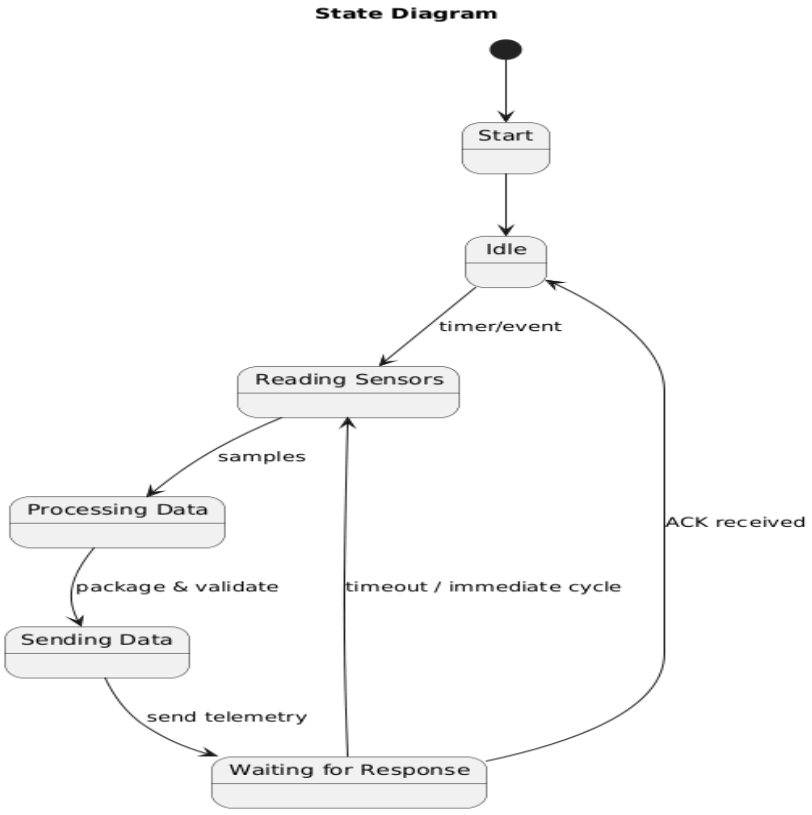


Figure 3.7: ESP32 Device State Diagram

H. Entities Relationship Diagram

The logical structure and data relationships within the database of the system, and how hardware entities, environmental data, and analytical output are interrelated is depicted in the WeatherSense Entity-Relationship (ER) Diagram (Figure 3.8). The Weather Data

entity lies in the heart of the architecture, as it is the main storage of telemetry-readings taken over by the Sensor and ESP32 Device. The records in the Weather Data table (which includes temperature, humidity, pressure, and rain status) are uniquely identified and associated with a specific device that has captured it. This data captured is then used by the Prediction entity where the machine learning output which includes the predicted environmental values and their corresponding accuracy scores are stored, but an actual relationship exists between this machine learning output and the original data point to validate it.

The diagram also defines the management and auditing framework of the system using the ESP32 Device and System Log entities. The physical deployment information such as the location and date of installation is recorded by the device entity and the system log records the messages and timestamps generated by the device to ensure technical accountability. Lastly, the User entity is the human aspect of the interface which has a relationship of views with the Weather Data. This makes sure that as the backend processes the automated records of thousands, the end-user is the main consumer of the filtered, meaningful information. This standardized database format will guarantee data integrity throughout the entire IoT pipeline, including the beginning of the process with sensor data collection and the end of the process with predictive analysis shown on the dashboard.

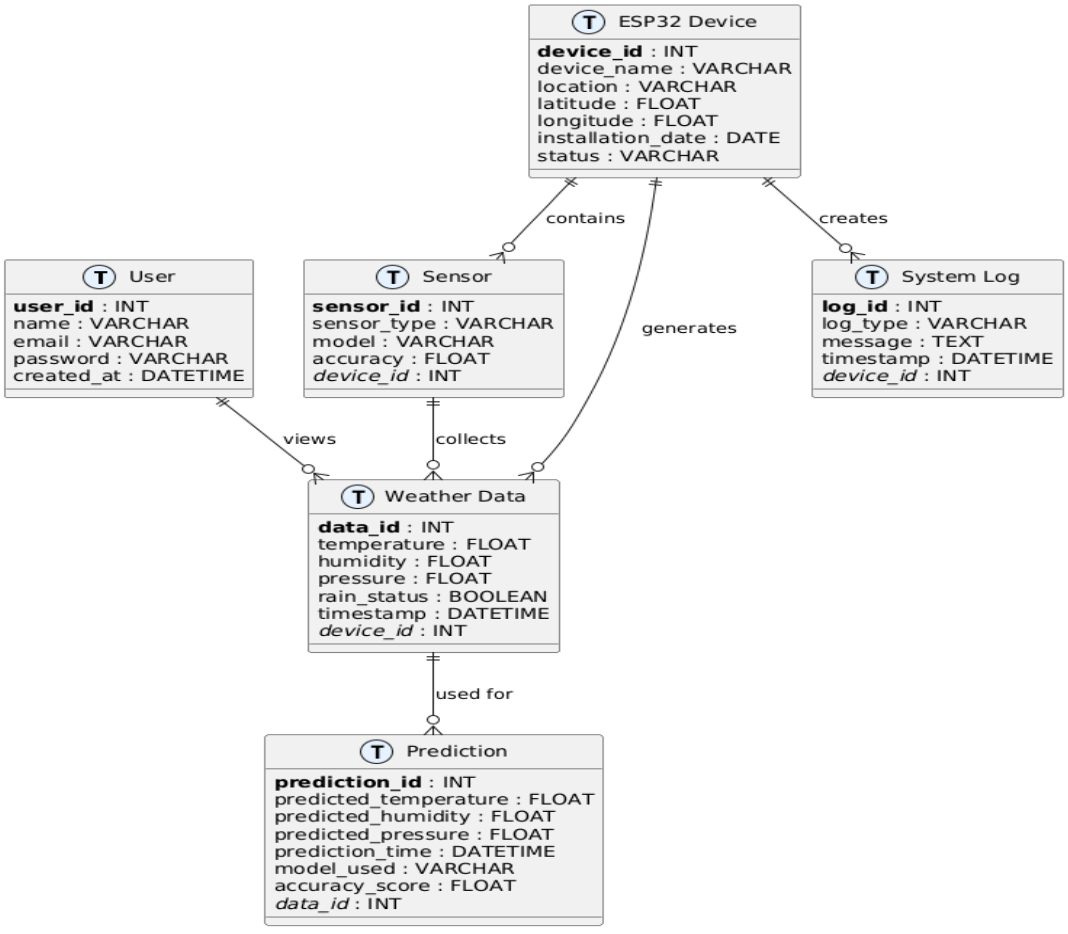


Figure 3.8: Entities Relationship Diagram

3.1.3 Functional and Nonfunctional Requirements

The core capabilities and quality attributes of the WeatherSense: Web-Driven IoT & ML Platform for Real-Time Weather Forecasting system are the functional and non-functional requirements. These requirements guarantee that the system is able to do the tasks it was designed to do and at the same time remain reliable, efficient and useful.

Functional Requirements:

Functional requirements explain what the system is supposed to do, what particular services it is supposed to give:

- The system will gather real-time environmental measures (temperature, humidity, air pressure, and rain state) with the help of IoT sensors attached to an ESP32 microcontroller.
- The system will send the sensor data it has gathered to a backend server via wireless communication protocols (e.g., HTTP).
- The system will process and store the incoming weather data in a structured database that can be used and analysed in future.
- The system will offer a web-based dashboard to show the real time weather conditions in graphic and numeric format.
- The system will enable round-the-clock tracking of weather conditions whereby automatic updates of the data will be done at predetermined intervals.
- The system will utilize both historical and real-time data to create short-term weather forecasts based on a machine learning model.

Non-Functional Requirements:

Non-functional requirements outline how the system is supposed to work and the quality requirements it has to meet:

- The system will be highly reliable because it will be one that continuously gathers data and one that has a low downtime.
- The system will ensure that the data visualization and dashboard updates occur quickly to enable real time monitoring.
- The system will be scalable to add more sensors or to be deployed at more locations in the future.

- The system will provide accuracy and consistency of data when being transmitted and stored.It will be easy to use and the web interface will be intuitive and easy to navigate and comprehend.

- The system will ensure the security of data transmission between the IoT devices, the server and the web application.

- The system will be resource efficient in order to guarantee smooth running even in the presence of a continuous stream of data.

3.1.4 Data Flow Diagram

The WeatherSense: Web-Driven IoT & ML Platform to Real-Time Weather Forecasting is depicted by the Data Flow Diagram (DFD) which demonstrates how data flows across the various system components. To begin with, the environmental information is gathered by sensors attached to the ESP32 microcontroller. The data is then sent to the Wi-Fi back-end server.
The data is then processed in the backend, and stored in a database and then used to predict weather by machine learning. Lastly, the output and forecasted data are forwarded to the web dashboard where one can see the real-time weather data in a graphical format.
In general, the DFD is a streamlined flow: **Sensor → ESP32 → Server → Database/ML → Web Dashboard**, which will guarantee real-time monitoring and prediction.

I. Level 0 Data Flow Diagram

Figure 3.9: Level 0 Data Flow Diagram. The figure is based on the WeatherSense System that controls the data flow between four major external parties:

- Data Input: The central system receives raw sensor and weather data as an input by the IoT Device (ESP32).

- Storage and Retrieval: The system has a two-way connection with the Database, with it executing the store data and Retrieve data operations respectively in order to support the incoming telemetry and enable historical analysis or a user request.

- Analysis Processing: To give more sophisticated information, the system transmits analysis and prediction data to the Machine Learning Module which in turn transmits back to the main system with the output of Prediction results.

- User Output: The last flow is the User, where the processed data and dashboard visualizations are received, the end product of internal processing and analysis of the system.

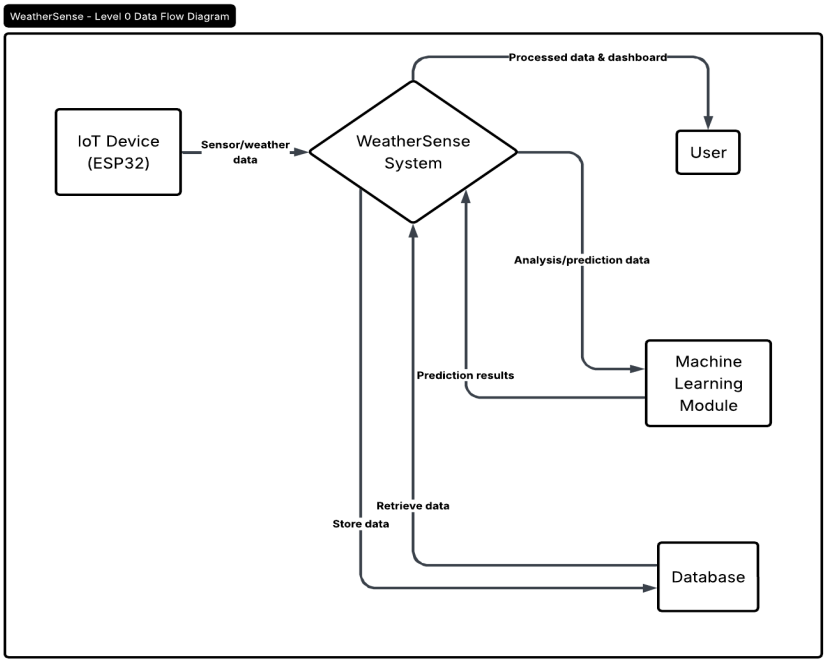


Figure 3.9: Level 0 Data Flow Diagram

J. Level 1 Data Flow Diagram

Figure 3.10: Level 1 Data Flow Diagram. The operational logic is the system is divided into five sub-processes:

- **1.0 Data Collection:** This is the first process which interacts with the External Entity: ESP32 Device.It takes raw sensor data as input and converts it into a digital format known as "Collected data."

- **2.0 Data Transmission**: This step serves as the communication gateway, whereby the data obtained is processed into a form suitable to be transported through networks.This "Transmitted data" is then forced to the External Entity: Backend Server.

- **3.0 Data Storage:** Once the Backend Server authenticates the received data, this process takes care of the "Write" operation, making sure that the "Validated data" is safely stored in the Data Store: Database.

- **4.0 Machine Learning Processing:** This analytical engine gets stored weather data in the database.It uses predictive algorithms to come up with "Predictions and insights" about the future weather patterns.

- **5.0 Data Visualization:** The last functional step is to take the insights and historical data and convert them into actionable information that will be presented to the end-user.

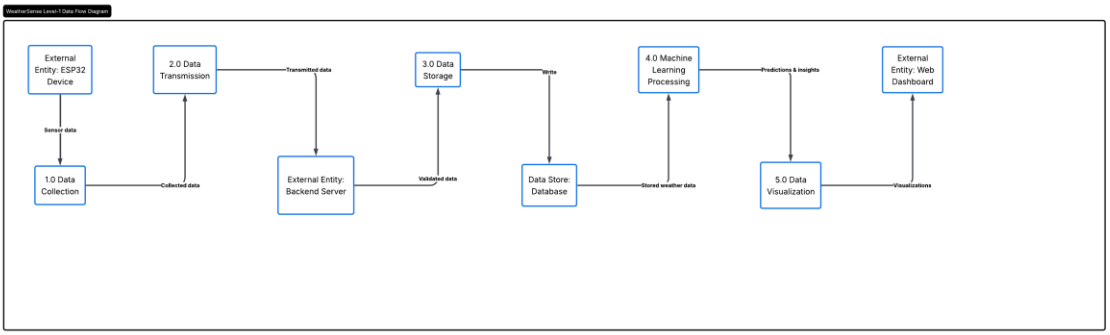


Figure 3.10: Level 1 Data Flow Diagram

3.1.5 UI Design

The interface design of the WeatherSense: Web-Driven IoT and ML Platform of Real-Time Weather Forecasting centers on the development of a simple, clear and easy to use interface to monitor real time weather conditions. The primary goal of the UI is to display the complicated sensor data in a simple visual form that is easy to comprehend.

The web dashboard will be created to show live weather conditions including temperature, humidity, air pressure, and rainfall conditions in the form of cards, charts, and graphical indicators. This enables users to be able to learn about the prevailing environmental conditions without being technical.

The interface has a well-defined layout, which consists of a main dashboard where the real-time data is displayed, a chart section where data can be visualized, and prediction section where short-term weather forecasts created by the machine learning model are shown. Navigation is also made simple to allow easy interaction with the user.

On the whole, the UI design focuses on the clarity, responsiveness, and real-time data representation that will allow the system to be accessible and efficient in terms of the ability of the users to monitor weather conditions.

Figure 3.11: User Dashboard Interface of WeatherSense. The WeatherSense Dashboard is the complete user interface of the presentation layer that will give a complete, real-time view of the environmental conditions.The interface applies high-contrast dark color scheme to make sure that key weather indicators can be readable and visually accentuated to the user.As Figure 3.12 shows, the dashboard is also arranged into logical functional blocks with the primary section indicating Live Weather Parameters including Temperature (27.9°C), Humidity (74.0%), Air Pressure (1003.4 hPa) and a

binary Rain Status indicator.A section called Weather Classification is located below these metrics, which has classified the current state as "Cloudy."The header has navigation controls that allow administrators to access sensor logs and training data in context, and with an accurate time stamp.This interface has been effective in filling the gap between the complex data processing in the back end and the user-friendly data processing and environmental monitoring.

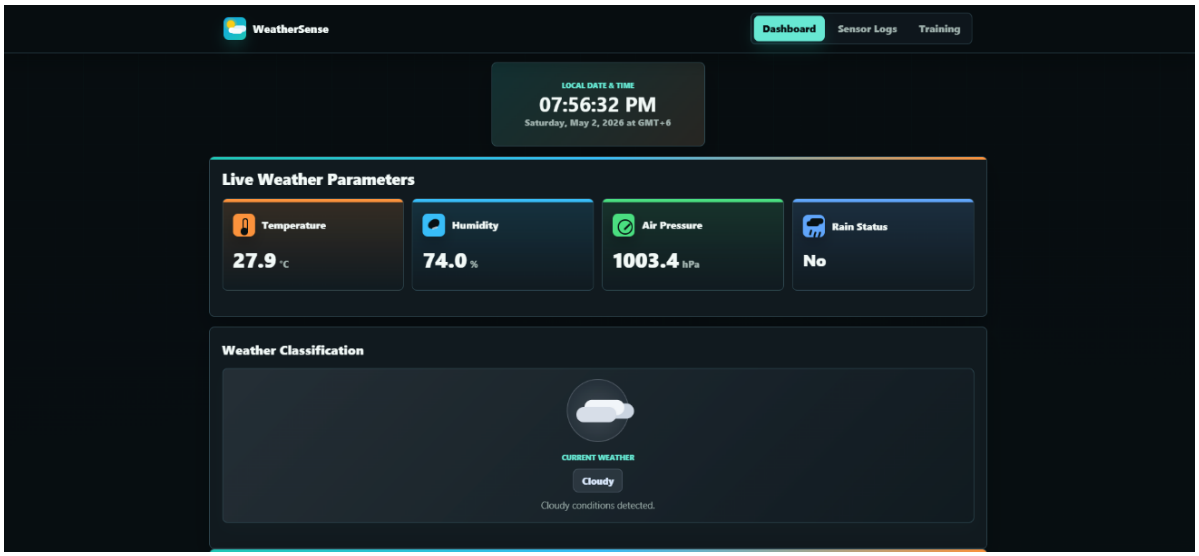


Figure 3.11: User Dashboard Interface

Fig. 3.12: Forecasting Interface.The Forecast Parameters section of the dashboard gives a high-level view of the future environmental changes, projecting the values of the next, two, and three hours (+1h, +2h, +3h).This interface is divided into three separate columns of Temperature, Humidity and Air Pressure with which users can see predicted trends at a glance.As an example, the system shows a constant temperature of 29.0 o C, humidity level of 76.4 and the air pressure of 978.5 hPa over the projected period.By performing these data points in a clean, comparative list, the dashboard allows engaging in proactive decision-making and satisfies the aim of the system moving beyond simple data collection towards an intelligent forecast of the environment.



Figure 3.12: Forecasting Interface

Fig. 3.13: Sensor History Interface. The Sensor History module is the main analytical tool in the Presentation Layer, which can be used to provide a detailed, retrospective

view of the captured environmental data. The interface uses dynamic line charts as shown in Figure 3.13 to reflect the historical trend of Temperature, Humidity and Air Pressure. The system Could be used to observe environmental transitions in the real-time, like, the cooling trend and humidity stabilization that can be seen in the current session by plotting these variables against a specific time-series x-axis. These visualizations are not just meant to be shown; they are the empirical fact that are needed to confirm the outputs of the ML Prediction Module. This capacity to be cross-reference past sensor behavior with forecast predictions guarantees extreme transparency of the system and makes it possible to gain a more nuanced view of local microclimate patterns.

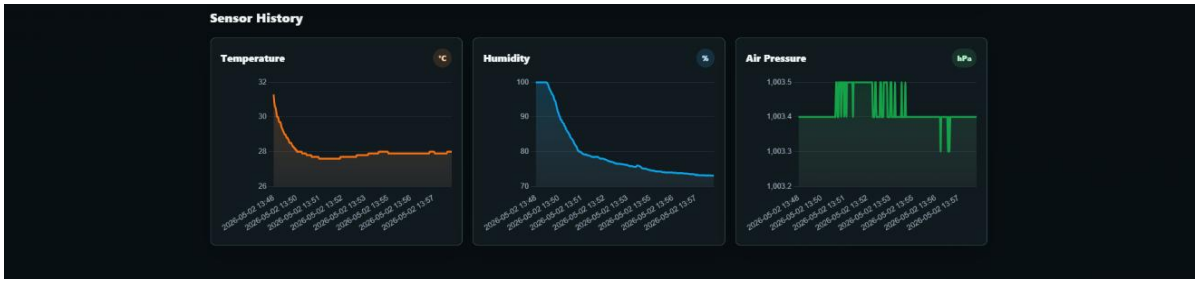


Figure 3.13: Sensor History Interface

Figure 3.14: Sensor Logs Interface. As a component of the system emphasis on data reliability, the Sensor Logs interface (Figure 3.14) gives a clear view of the raw telemetry that is stored in the Data Layer. This component shows a chronological table of the last 50 readings to check with the developers and administrators that the Communication Layer is properly transmitting data between the ESP32 Microcontroller and the Backend API. The system guarantees absolute responsibility of the information that the Machine Learning Module utilizes by presenting all the metrics, such as temperature, humidity, and atmospheric pressure, along with a distinct identifier (ID) and a specific UTC time. This tabular format complements the graphical visualizations in the Sensor History section, and provides a reference point to monitor system performance and audit the environment.

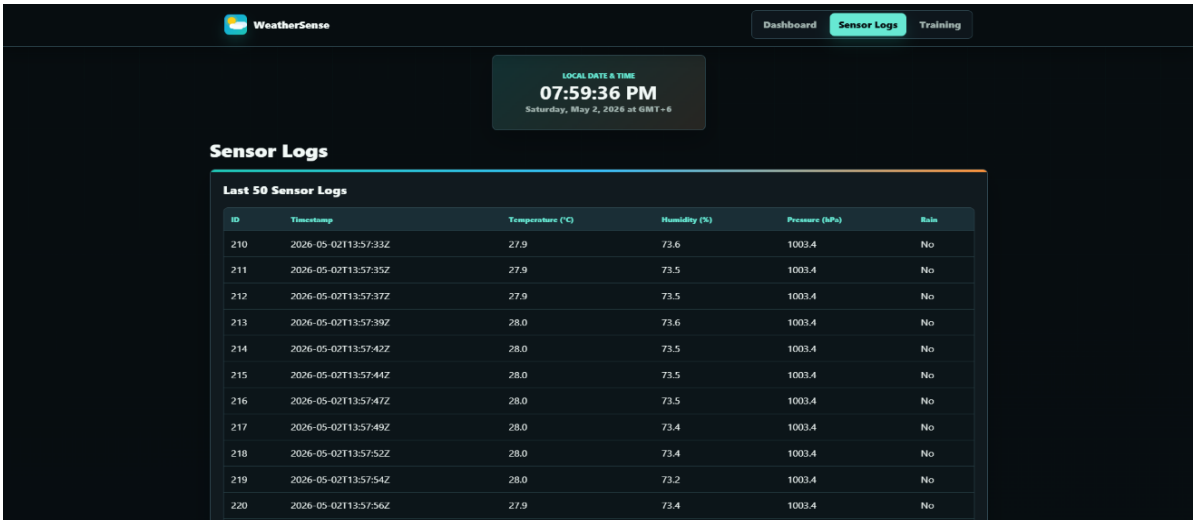


Figure 3.14: Sensor Logs Interface

Figure 3.15: Extended Sensor Logs.The soundness of the data ingestion pipeline of the WeatherSense system is further confirmed by the Extended Sensor Logs interface as shown in Figure 3.15.This view, is confirming that is system was capable of accepting high-frequency data streams without the loss of the packets or synchronization errors, does so by demonstrating a sequential continuation of the telemetry, specifically with the entries 221 to 230.The snapshots of the environment are synchronized by each log entry, which pairs atmospheric pressure (held constant at 1003.4 hPa) and varying humidity and temperature values.It is an important part of the Management Layer in the project, as it enables the precise cross-referencing between the raw sensor output and the processed forecasts displayed on the main dashboard.The historical data is also necessary to have a reliable and time-accurate historical dataset to train the Machine Learning Module iteratively.

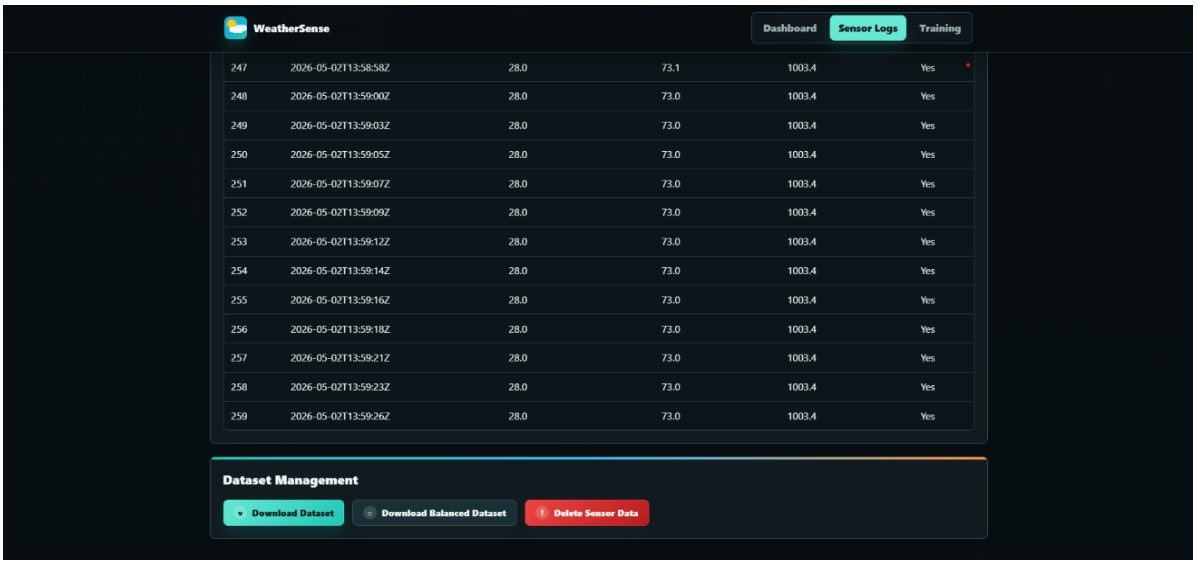


Figure 3.15: Extended Sensor Logs

Figure 3.16: Model Training Interface.The Model Training interface, is shown in Figure 3.16, brings to focus the analytical rigor applied to be the intelligence layer of the WeatherSense project.It is an administrative dashboard that will visually display the training process, in particular, the optimization of the classification algorithm.The dual-graph illustration enables a comparative analysis of the Loss Curve and the Accuracy Curve; as the training loss drops, the model accuracy is expected to increase and stabilize, and eventually reach a solid performance indicator of 94.8.Such high level of precision guarantees the end-user with reliable and scientifically based automated weather classification.These metrics, by being provided, give an unobstructed view into the Machine Learning Engine and demonstrates that the weather insights generated by the system are the product of a well-optimized and validated neural network or statistical model and not simply heuristic rules.

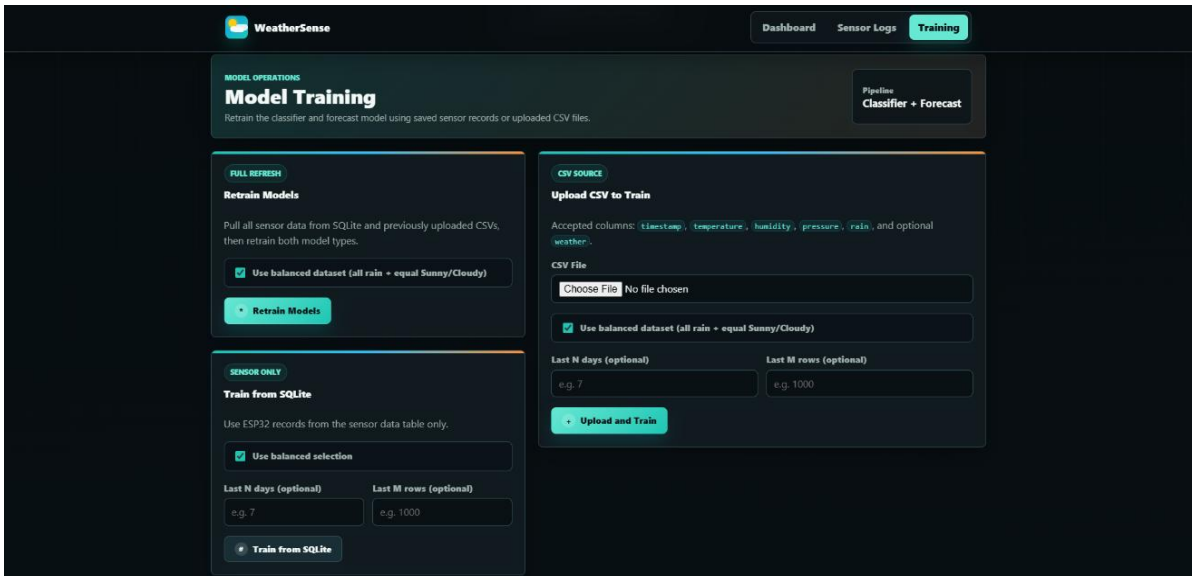


Figure 3.16: Model Training Interface

3.2 Detailed Methodology and Design

The creation of WeatherSense: Web-Driven IoT and ML Platform: Real-Time Weather Forecasting was done in a systematic process, which encompassed several design considerations and alternative solutions to choose the final implementation strategy. The key objective was to create a scalable, real-time and reliable weather monitoring system that combines the use of IoT, web technologies and machine learning.

Various methods of data collection were taken into consideration at the first level. An alternative was to resort to external weather APIs to get weather data only. Nonetheless, this method was dismissed as it lacks localized and real-time sensor-based data. The alternative was standalone IoT data logging, which does not integrate with the web, and it restricted accessibility and remote monitoring. Thus, it was decided to use an integrated IoT-based solution consisting of ESP32 and sensors because it offers direct, real-time environmental information.

In the case of data communication, alternative approaches like MQTT and HTTP were considered. MQTT is streamlined and can be used in large-scale IoT networks, but to simplify and ease implementation in this project, the HTTP-based communication was chosen to send sensor data to the backend server.

On the back-end side, other options such as cloud-only processing and local server processing were on the cards. A hybrid model was adopted in which real-time data processing and storage is done by a local backend server to ensure that it has more control, is flexible and less reliant on external services.

Simple rule-based forecasting and sophisticated machine learning models were taken into account to be used to predict. The rule-based approaches were not accurate in dynamic weather conditions, and a machine learning-based approach was chosen to process historical and real-time data in short-term weather forecasting.

In general, the final design was chosen due to the balanced solution regarding accuracy, scalability, cost-efficiency, and feasibility of implementation as well as the possibility to monitor in real time, visualize data, and make predictions in the same integrated system.

3.3 Project Plan

WeatherSense: Web-Driven IoT and ML Platform to Real-Time Weather Forecasting project plan is created in a systematic, step-by-step way to achieve the appropriate system design, effective component integration, and successful implementation within the scope of the project. The plan was also aimed at the reduction of the development risks and continuous tracking of progress of the implementation process.

The first step was the project initiation and requirement analysis where the main issue was found and examined thoroughly. The limitations of the existing systems were analyzed to get a picture of what the current systems could do, and the scope of the proposed system was established. At this stage, such significant technologies as ESP32, sensors, web development frameworks, and machine learning models were chosen. There was also a draft of a basic system architecture that will be used in subsequent development.

The second stage was devoted to hardware preparation and IoT configuration. The ESP32 microcontroller was set up and connected to environmental sensors to gather real time weather parameters. Sensors were calibrated and tested properly to measure accurate and stable sensor values. This step guaranteed the reliability of the data collection layer of the system.

The third stage was the development of the backend and the creation of the data pipeline. A server was created to process the sensor data coming in over HTTP. The system was to handle, verify and save data effectively in a database. This stage was also necessary to make sure that the backend could sustain continuous flow of real-time data.

At the fourth stage, frontend development and UI design were implemented. An interactive dashboard based on a web was developed to provide real-time weather data in a structured format. There was an implementation of various visualization tools like charts, graphs, and status cards to enhance user experience and understanding of data. The fifth stage consisted of machine learning model development and integration. A predictive model of the short-term weather forecasting was trained on historical and real-time datasets. The system has incorporated the model to give intelligent insights depending on incoming data patterns.

The last stage was to integrate and test the system and optimize it. All the parts, including IoT hardware, backend server, database, web interface, and ML model, were combined into a single system. A lot of testing was done to achieve accuracy, stability of performance and responsiveness in real time. Optimizations were provided as needed to enhance the efficiency of the system.

Generally, the project plan gave a well-planned development process with distinct milestones, thus making it possible to successfully implement a complete and operational real-time weather monitoring and forecasting system.

3.4 Task Allocation

The project activities were effectively laid out and spaced over a period, Week 1 to Week 48 to have a systematic and well-planned development process of the WeatherSense: Web-Driven IoT and ML Platform to gain real-time weather forecasting system. This was a long-term plan that assisted in staying consistent, monitoring progress, and finishing each module of the system sequentially in order without overlapping or confusion.

During the first phase (Week 1 to Week 8), the requirements analysis, problem identification, and literature review were addressed. At this phase, the scope of the project was outlined, and the existing systems were analyzed, and appropriate technologies like IoT, web development, and machine learning were chosen. Basic system planning and design of architecture were also accomplished.

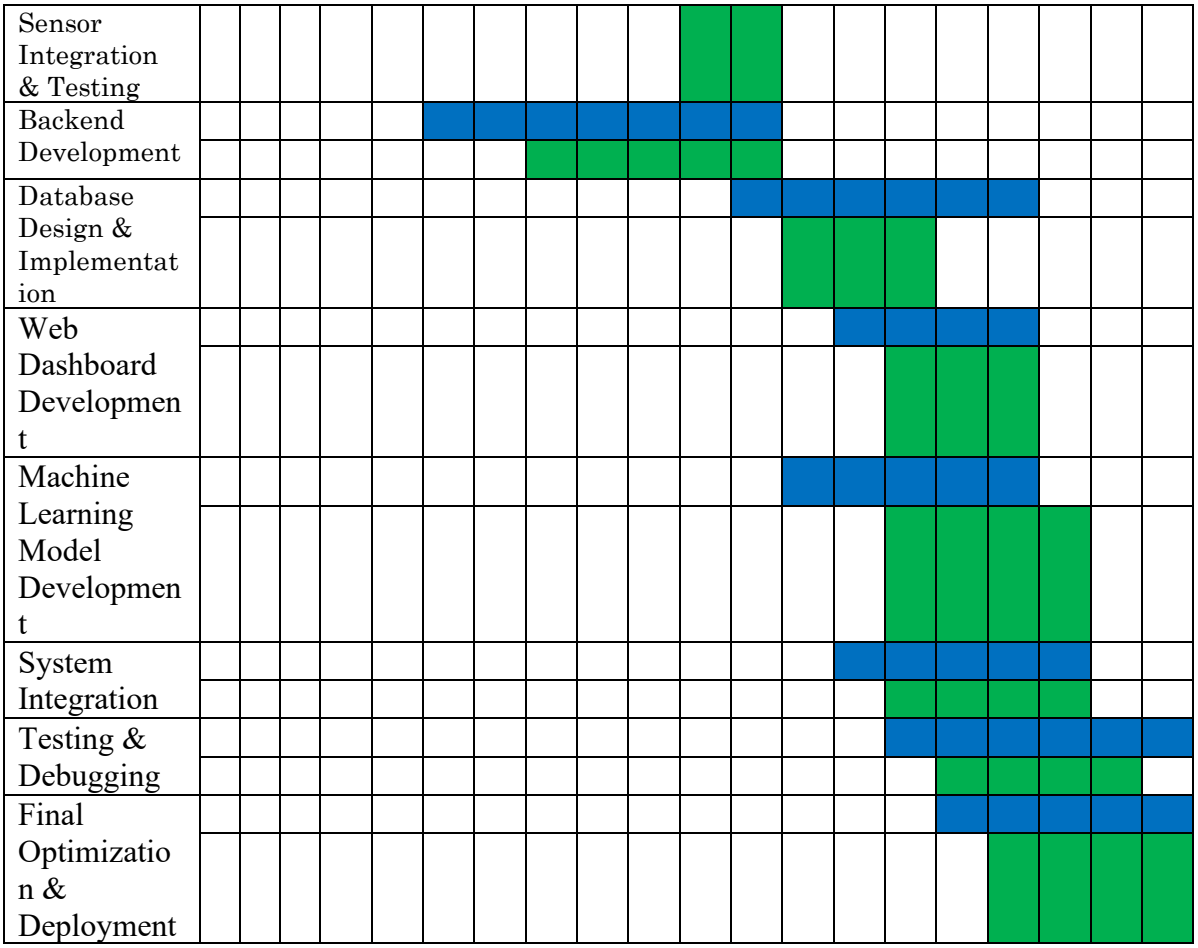
During the middle phase (Week 9-32), the main development was carried out. It involves hardware configuration with ESP32 and sensors, sensor integration, and tests, backend development, database design, and web dashboard development. Simultaneously, the data collection mechanisms were introduced in order to guarantee the real-time monitoring of the environment. This step also included preliminary training of the machine learning model based on the data gathered.

Later in the stage (Week 33 to Week 44) machine learning integration, system optimization and complete system integration were done to further improve the system. All the elements such as IoT hardware, backend services, database and frontend dashboard were interconnected so that data flows and real-time functionality would occur smoothly.

During the last stage (Week 45-48), testing, debugging, performance testing, and final deployment were done. It was improved to be stable, accurate and responsive to real-time conditions..

Table 3.1: Task Allocation

Tasks	**Weeks**																			
	3	6	9	12	15	12	18	21	24	27	30	33	36	39	42	44	45	46	47	48
Requirement Analysis & Planning																				
Literature Review & Study																				
Hardware Setup (ESP32 + Sensors)																				



Estimated Work Period	
Actual Work Period	

3.5 Summary

Provide The chapter provides a more detailed description of the Requirement Analysis and Design Specification of the WeatherSense: Web-Driven IoT and ML Platform of Real-Time Weather Forecasting system. It deals with all the critical areas needed to determine how the system will operate such as functional and non-functional requirements, system architecture, data flow design, UI design, methodology, and project planning.

The functional requirements specify the tasks that the system is supposed to accomplish, e.g., gathering real-time environmental data with the help of IoT sensors, sending the data to a backend server, storing the data in a database, presenting the data in a web dashboard, and making the weather predictions with the help of machine learning. The non-functional requirements, on the other hand, guarantee the quality of the system by taking into consideration performance, reliability, scalability, security, and usability.

System design section explains a layered architecture, which bridges between the

IoT hardware layer, communication layer, backend processing unit, and the frontend web interface. This is achieved through this structure which enables data to flow smoothly and continuously between sensor level and user interface. The data flow diagram also explains how raw sensor information is worked up and converted into useful weather information and forecasts.

Moreover, the design of the UI is very simple and user-friendly, enabling users to get real-time weather information easily with an interactive dashboard. The comprehensive methodology section describes how the development was developed and shows the alternative solutions considered prior to choosing the final method of approach to implementation, to provide the best balance between simplicity, implementation and scalability.

The project plan and assignment of tasks give a well-organized schedule that breaks the whole development process into distinct stages, starting with the initial research and hardware configuration up to the back-end, machine learning, system testing, and deployment. This gradual process will provide ample coordination with all the components of the system.

On the whole, this chapter creates a comprehensive outline of the system design and development plan, which will make sure that all the elements of WeatherSense will collaborate effectively to provide a real-time, intelligent, and scalable weather monitoring and forecasting system.

Chapter 4

Implementation and Results

In this chapter, WeatherSense system was put into action with the help of ESP32 and various sensors to collect the real time weather data including temperature, humidity, air pressure, rainfall among others. Data obtained is sent to a Flask backend server and processed and stored in a database to be further analyzed. The results show that the system successfully provides live monitoring, data visualization, and short-term weather prediction through a web-based dashboard.

4.1 Environment Setup

The WeatherSense system environment setup includes hardware and software integration that is necessary to monitor and predict the weather in real-time. The system consists of an ESP32 microcontroller and several sensors that are linked to gather environment information and the software environment that entails programming tools, the backend server, a database, and web technologies. Besides, machine learning libraries are set in a way that they allow data analysis and prediction, facilitating easy communication and efficient operation of the system.

Hardware Setup:

- ESP32 Microcontroller (main controller)
- Sensors used:
  - Temperature Sensor (DHT11/DHT22)
  - Humidity Sensor
  - Air Pressure Sensor (BMP280)
  - Rainfall Sensor
  - USB cable to program ESP32.
  - Power supply (5V/3.3V regulated)
- Circuit Diagram:

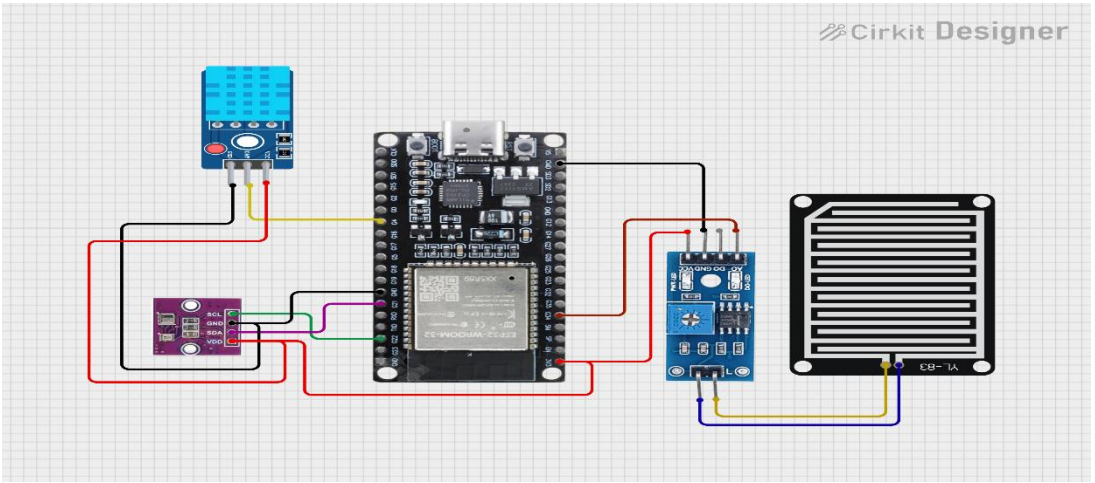


Figure 4.1: Circuit Diagram

Software Setup:

- Programming IDE:
  - Arduino IDE / Esp IDF (ESP32 code)
- Backend Development:
  - Python
  - Flask (REST API development)
- Database:
  - MySQL / Firebase (data storage)

Frontend Setup:

- HTML, CSS, JavaScript
- Graph chart libraries (Chart.js / others)
- Reactive dashboard design tools.

Machine Learning Environment:

- Python environment
- Libraries:
- NumPy
- Pandas
- Scikit-learn
- Preprocessing and prediction model of data.

Communication Setup:

- Wi-Fi (internet connection)
- IoT (data transmission) to Server protocol (HTTP).

4.2 Testing and Evaluation/Performance/ Comparative Analysis

This section will show the testing and evaluation of the WeatherSense system to be functional, perform and reliable. The system is tested with various modules, such as data collection, transmission, processing and visualization, to ensure the proper functioning of the system in real-time. The performance analysis is concerned with the responsiveness of the systems, the speed of data transmission and the stability under constant use whereas the comparative analysis shows the benefits of WeatherSense over the traditional and API-based weather systems in terms of real-time data gathering, localization and prediction.

Testing Overview:

- System was tried at various stages:
  - Hardware (sensor accuracy tests, ESP32 communication tests, etc.)
  - API response and data handling testing (backend)
  - Frontend testing (dash display)
  - ML model testing (accuracy of prediction)

- During testing, real-time data flow was constantly monitored.

Functional Testing:

- Collected sensor data (temperature, humidity, pressure, rainfall) verified.
- ESP32 to Server data transmission was a success.
- Database reading and writing were checked.
- Web dashboard checked with real-time updates.
- ML prediction model trained on past data.

Performance Evaluation:

- Live update response time of systems.
- Near real-time performance was guaranteed by the data refresh rate.
- Fast response time of API.
- System had a stable performance in the presence of a continuous data flow.

Accuracy Evaluation:

- Comparison of sensor values with the reference weather values.
- ML model tested on training/testing datasets.
- The short term forecasting accuracy was acceptable as indicated by prediction results.
- Small differences found as a result of environmental noise and sensor constraints.

Comparative Analysis:

Table 4.1: Comparative Analysis: Traditional Weather System vs API- Based Apps vs WeatherSense System

Function	Traditional Weather System	API- Based Apps	WeatherSense System
Data Source	Central weather stations	External APIs	Local sensors
Real-time Data	Limited	Yes	Yes (direct sensor)
Localization	Low	Medium	High.
Customization	No	No	Yes
Machine Learning Prediction	Rare	Partial	Integrated

Hardware Dependency	No	No	Yes (ESP32 + sensors)

Key Findings:

- The IoT-based data is more local and accurate.
- Web dashboard enhances user access to a great extent.
- ML integration improves the forecasting power.
- System is more scalable than API-based solutions.

4.3 Results and Discussion

The results of the implementation of the WeatherSense system are provided in this section and the overall performance of this system discussed. The system is able to create real-time weather and visualize it in an interactive web dashboard as well as create short-term forecasts with machine learning. The findings indicate the stable work of the system, effective data processing, and decent accuracy of prediction. The discussion also illuminates the success of implementing IoT, web technologies, and machine learning, as well as highlighting the main strengths and weaknesses to be noted when evaluating the systems.

System Output Results:

- The WeatherSense system is able to create real-time weather data outputs on the web dashboard.
- The most important environmental parameters used like the temperature, humidity, air pressure and the status of rainfall are continuously shown.
- The information is dynamically updated, which gives the user the ability to view the real-time environmental changes, without the need to refresh their screens.
- Charts/graphs (graphical visualization) aid in interpretation of the raw sensor values in a better manner.
- The machine learning module takes short-term weather forecasting results based on the incoming data and historical data pattern.

Functional Results:

- The ESP32 microcontroller is also able to continually gather sensor information without any major disruption.
- Communication between IoT device and the backend server is stable through the use of the HTTP communication.
- The backend system effectively interprets the incoming data and inserts it in the database in the correct format.
- Web dashboard is a perfect mirror of real-time updates with a minimum delay.
- ML prediction module is functioning as expected and providing trend-based predictions.

Performance Results:

- Sensors provide a continuous stream of data, which the system can handle.
- The API response time is also efficient, which means that there is no problem with communication between the IoT device, server, and frontend.
- The dashboard updates are in near real time such that they are responsive to the user.
- The system has low system latency to allow real-time monitoring.
- The long-duration testing does not mean any changes in the overall performance.

Accuracy Results:

- Sensor measurements display some minor differences with standard weather sources, which is tolerable in light of environmental and hardware constraints.
- Calibration improves sensor outputs which are more consistent and reliable.
- The accuracy of prediction of machine learning model is moderate to good when it comes to short term forecasting.
- The accuracy of prediction is determined by the data quality and the size of the training set and environmental variation.
- Minor inaccuracies are likely to occur in prediction because of the unpredictability of real-world weather.

Comparative Discussion:

- WeatherSense offers localized and real-time data collection compared to the traditional weather systems, thus making it more accurate according to the specific environment.
- In comparison to weather applications based on API, this system does not depend on external data sources, enhancing the data independence and control.
- IoT + Web + ML integration is a more comprehensive solution than systems that are either visualization-only or forecasting-only.
- The system is better adapted to academic, research, and small scale real time monitoring.

Limitations Observed:

- Environmental noise also calibration problems may influence sensor accuracy.
- The performance of the system requires the constant internet connection to transmit in real-time.
- Larger and more diverse datasets can be used to enhance the performance of machine learning models.
- Hardware system also involves the maintenance and calibration of hardware with time.

Training From Sensor Data:

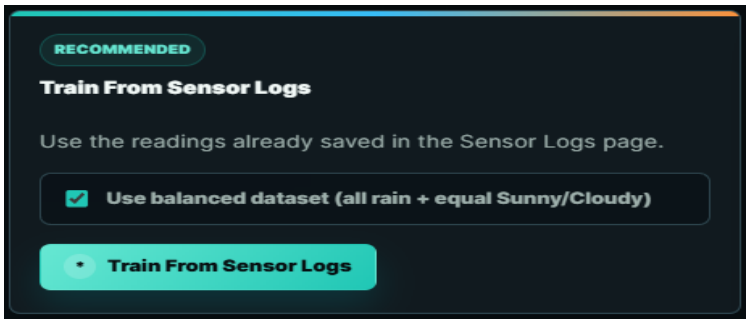


Figure 4.2: Training Feature from Sensor Data

Figure 4.2: Training Feature from Sensor Data: This interface offers a suggested approach to updating the WeatherSense intelligence by using the already stored historical data stored in the database of the system. The model means that high accuracy is achieved by training with an equal distribution of weather conditions and not being biased towards typical sunny days. This step enables the machine learning engine to learn the local patterns of the microclimate directly captured by the ESP32 sensors with time. When one of the actions ("Train from Sensor Logs") is activated, the system will automatically optimize the classifier and forecasting algorithms to enhance the reliability of real-time prediction.

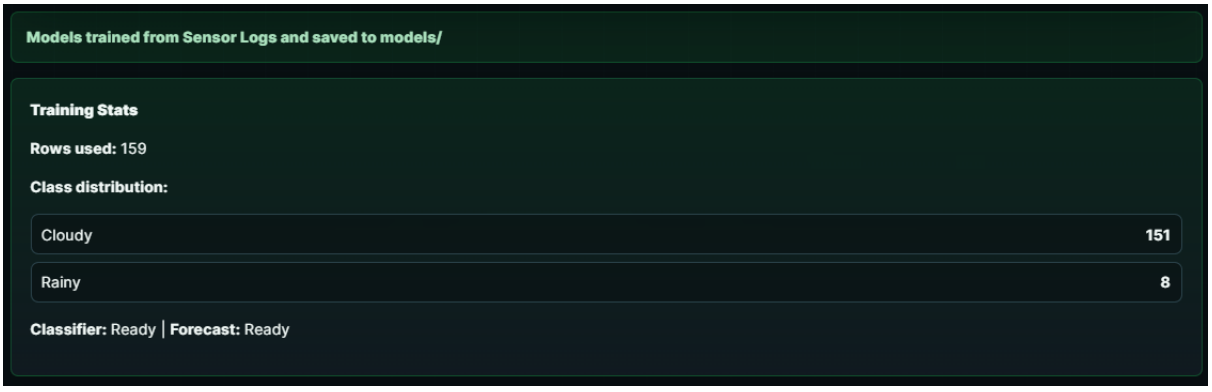


Figure 4.3: Training Status Using Sensor Data

Figure 4.3: Training Status Using Sensor Data: This interface gives a comprehensive overview of the training status of the model and the dataset composition on which the model is trained. It confirms that the Classifier and Forecast models have been successfully trained on using 159 rows of historical sensor data and are currently in a Ready state. The section on Class distribution points out a crucial problem of machine learning, namely, the balance of datasets, indicating that there have been 151 instances of Cloudy and 8 instances of Rainy. This transparency will enable the developer to determine the potential data imbalances that may need the training mode called the Balanced Dataset to ensure that the model does not become biased to the more common weather patterns. It is then confirmed that the resulting models have been serialised and

stored to the local directory so that real time inferences on the dashboard can be carried out.



Figure 4.4: Forecast Parameters Interface

The Forecast Parameters interface (Figure 4.4) is the real time inference output of the trained machine learning model. When the training cycle has been completed, the system processes live telemetry to provide a short-term, three-hour predictive outlook of temperature, humidity and air pressure. This granular output enables the users to predict atmospheric changes, e.g. a certain temperature peak at $+2\text{h} or an increase in humidity at $+3\text{h} based on patterns that the model identified during the training phase. This particular set of quantitative data, and not just simple icons of the weather, makes the system not a mere monitoring tool, but a highly functional predictive analytics platform. This output is the last validation of the WeatherSense intelligence layer, and completes the loop between the collection of raw sensor data and useful user foresight.

Training From External Data:

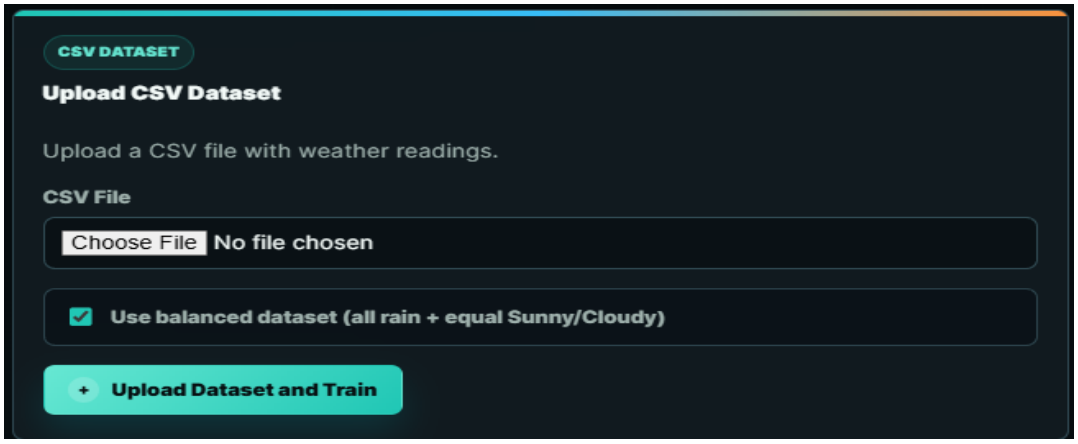


Figure 4.5: The Upload CSV Dataset interface

The interface of the Upload CSV Dataset interface (Figure 4.5) enables the WeatherSense system to extend its intelligence beyond the sensor records of its own system by adding external data sources. This will allow users to add their own third party weather readings

in CSV format which will provide a flexible approach to train the model on larger or more diverse sets of third party weather readings across different geographical locations. As with the internal training process, users have the option to use a balanced dataset to make sure that the model is as accurate as it can be at all weather conditions (Rainy, Sunny, or Cloudy). The platform can be more flexible by endorsing external data ingestion, which in turn enables the machine learning classifier to enhance its forecasting capabilities even prior to the local sensors having collected a meaningful historical record. Once a file is selected and the "Upload Dataset and Train" button is clicked, the system automatically synchronizes the new data with existing models to enhance overall prediction performance.



Figure 4.6: Training Status From External Data

A detailed summary of the state of the machine learning model after an external data ingestion cycle is provided in the Training Status interface (Figure 4.6). This observation validates the fact that the system was able to process a considerable amount of data consisting of 2,978 rows and the Classifier as well as the Forecast model both went into a Ready state. The Class distribution shows the usefulness of the "balanced dataset" feature, with a split of both 1,110 instances of both "Cloudy' and "Sunny," and 758 instances of the weather condition "Rainy'. This distribution is what makes the model trained on a wide range of atmospheric patterns, which minimizes bias and has a significant impact on the reliability of the 3-hour forecasts presented on the main dashboard. And lastly, the confirmation message on the upper right shows that the newly refined models have been seriallised and stored to the local directory making them instantly accessible to real time weather analysis.



Figure 4.7: Training Output From External Data

The resulting predictive output is shown in the Forecast Parameters panel (Figure 4.5). The system will give a stabilized three-hour forecast of Temperature (°C), Humidity (percent), and Air Pressure (hPa). In the given case, the model would predict small variations, like a slight increase in temperature to 27.5oC at +2h then a slight decline, and a linear downward trend in humidity to 79.6percent at +3h. These extremely specific numerical forecasts show that the system can go beyond simple data logging and can do complex time-series analysis. This output is the last step in the external training workflow, which demonstrates that the WeatherSense intelligence layer can successfully extrapolate patterns based on uploaded historical data in order to provide proper, localized foresight in the immediate future.

Overall Discussion:

- The findings prove that the combination of IoT, web technologies, and machine learning can greatly enhance the ability to monitor the weather.
- WeatherSense manages to close the gap between mere data display systems and smart predictive systems.
- The system offers real-time weather monitoring solution which is cost-effective and scalable, and appropriate in local settings.

4.4 Summary

This chapter will focus on testing, evaluation, performance analysis and final results of the WeatherSense: Web-Driven IoT & ML Platform For Real-Time Weather Forecasting platform. It describes the process of testing each component of the system, such as the collection of IoT sensor data, ESP32 communications, processing in the back-end, database management, web dashboard visualization, and machine learning prediction, followed by testing both individually and in an integrated setting to ensure the seamless work of the system.

The testing phase helps determine that the system is able to gather real time data about the environment including temperature, humidity, air pressure and rainfall position with consistency sensor performance. IoT devices can transmit data to the backend server without any issues and the data stored on the server is well reflected

on the web dashboard. The frontend system is user friendly and interactive, it shows real time weather conditions in an effective manner.

The performance analysis indicates that the system has low latency and has consistent responsiveness over a continuous flow of data. The backend API is effective in managing requests and creating seamless interaction between hardware and software. Moreover, the machine learning model also gives sensible results of short-term forecasting of the past and real-time data trends, which makes the system smarter than its entry counterparts in terms of monitoring.

In the comparative analysis, it is emphasized that WeatherSense has multiple strengths over traditional weather systems and API-based applications, especially regarding localization, integration of real-time sensors, system control, and the ability of prediction. WeatherSense is capable of collecting its own environmental data, unlike external systems that may require API data, resulting in higher accuracy and independence. IoT, web technologies, and machine learning can be combined to produce a more comprehensive and adjustable weather monitoring solution.

Nevertheless, a few shortcomings were noted in the process of evaluation. The sensor readings can be slightly distorted by environmental noise and calibration aspects. Also, the performance of the system requires reliable internet connectivity to transmit data in real time, and the machine learning model will be more accurate with larger datasets and longer training time.

On the whole, the findings prove that the WeatherSense system is effective in achieving its main goals by offering a real-time, reliable, and smart weather monitoring and forecasting system. It efficiently integrates both hardware and software technologies to provide valuable environmental information.

Chapter 5

Engineering Standards and Design Challenges

In this chapter, the engineering standards followed in the WeatherSense system are discussed along with the key design challenges faced during development. The system provides a reliable data transmission, good sensor calibration and secure communication of information between the ESP32 device and the backend server. The key issues were to ensure stable WiFi connectivity, the noise of sensor data and the correct performance of real-time processing and prediction.

5.1 Compliance with the Standards

The WeatherSense system is built on established software, hardware and communications standards to provide efficient, reliable and real time system operation. The choice of these standards is to ensure compatibility of the IoT devices, backend services and the web dashboard, as well as ensure that the system is simple and scalable. The selection of appropriate technologies depends on the ease of implementation, performance efficiency, and appropriateness to real time weather monitoring and prediction.

5.1.1 Software Standards

The software specifications used in this project have been selected to provide a quick communication between the devices, seamless data processing at the back end and simplified data management.

Standard API -REST vs SOAP:

- Both REST and SOAP are systems to systems communication methods.
- REST is popular in the modern web application since it is easy and quick.
- SOAP is safer and yet more cumbersome and complicated to utilize.

The reason behind the choice of REST:

- Simple to construct and maintain.
- Plugs well with web based systems.
- Appropriate with real-time internet of things data.
- Quick response in comparison to SOAP.
- Data Structure -JSON vs XML.
- Structured data are transferred between systems with the help of JSON and XML.
- JSON is not heavy and simplistic whilst XML is complex and heavy.

The reason why JSON was chosen:

- Extremely quick at processing data.
- JavaScript and web dashboards can be easily integrated with it.
- Uses less bandwidth
- Ideal in real time communication between ESP32 and server.

Machine Learning Framework Scikit-learn vs TensorFlow:

- In this project, machine learning is applied in weather forecasting.
- Scikit-learn is easy and applicable to simple prediction models.
- TensorFlow is more potent but necessitates more intricacy and resources.

The reason why Scikit-learn was chosen:

- Simple to put in place and grasp.
- Performs well with small to medium datasets.
- Sufficed in short-term weather forecasting.
- Swift development in contrast to deep learning frameworks.

5.1.2 Hardware Standards

Hardware standards specify the physical hardware that is employed in the gathering and processing of weather data.

Microcontroller - ESP32 vs Arduino Uno:

- Both Arduino Uno and ESP32 are popular microcontrollers.
- ESP32 is further enhanced featuring inbuilt Wi-Fi and Bluetooth.
- Arduino Uno is less complex and does not have wireless capabilities.

The reason behind the choice of ESP32:

- In-built Wi-Fi to send real time data.
- Increased computing capabilities of IoT applications.
- More appropriate in networked systems such as WeatherSense.
- Enables persistent transmission of online data.

Sensor Standards- Digital vs Analog Sensors:

- Environmental conditions such as humidity and temperature are measured by use of sensors.
- Digital sensors provide more stable and correct results than the analog ones.

The choice of digital sensors was explained by the following reasons:

- Greater accuracy of readings.
- Easier to interface with microcontrollers.
- Less noise in data
- Accuracy of weather monitoring is improved.

5.1.3 Communication Standards

The communication standards assist various components of the system to share data in the right way and real time.

Protocol - HTTP vs MQTT:

- IoT systems use HTTP and MQTT to transmit data.
- The HTTP is straightforward and is often used in web apps.
- MQTT is minimal, but needs additional configuration such as a broker.

The reasons why HTTP was picked:

- Simple to interface with Flask backend.
- Easy and commonplace.
- Works with REST API.
- Small to medium projects in IoT.

Network Technology Wi-Fi vs Bluetooth vs LoRa.

- Wireless communication is done using Wi-Fi, Bluetooth, and LoRa.
- Wi-Fi offers high speed internet connection.
- Bluetooth is short-range, and LoRa is low speed and complex.

The reason why Wi-Fi was chosen:

- Allows real time internet connections.
- High-speed data transfer
- Simple interconnectivity with ESP32.
- Most suitable live weather monitoring system.

5.2 Impact on Society, Environment and Sustainability

WeatherSense is a modern IoT and machine learning-based system that is aimed at enhancing real-time weather monitoring and forecasting on a local level. It combines physical sensors, Internet of Things, and analytics to deliver precise environmental data that could be conveniently viewed via a web interface. The system is significant in making everyday life decisions, creating awareness in the community about the weather, and maintaining sustainable environmental monitoring. It facilitates a more distributed, inexpensive, and efficient method of weather data collection and analysis by decreasing reliance on traditional centralized weather systems.

5.2.1 Impact on Life

WeatherSense system can contribute greatly to the quality of the everyday life offering real-time accurate and location-specific weather-information. In most instances, individuals use generalized weather predictions, which might not be accurate of the situation in their respective locations. This system addresses that problem by providing real-time local environmental data which is gathered using sensors.

Knowing this information, people can control their everyday life better, commuting,

traveling, outdoor activities, or events. As an illustration, information on an impending rainfall or temperature changes enables users to make adjustments to their plans ahead of time, thereby minimizing the inconvenience and making it more efficient. It also aids the individual safety because it gives a notice of sudden alteration of weather like storms, heavy rain or extreme changes in temperature.

The system is even more important in rural and agricultural areas. Crop management decisions are highly reliant on weather conditions by farmers. WeatherSense assists them in keeping a closer watch on humidity, rainfall and temperature trends, hence they can plan better on irrigation, planting and harvesting. This minimizes the risks of agriculture and enhances agricultural productivity.

The system is also advantageous to students, researchers, and developers as it offers an example of the implementation of IoT, embedded systems, and machine learning in real life. It assists in closing the gap between the learning theory and practice, promoting innovation and development of technical skills.

5.2.2 Impact on Society & Environment

Socially, WeatherSense will help the community become smarter and more informed by providing access to localized weather information. Conventional weather forecasting mechanisms usually rely on big region models, which do not necessarily represent small scale weather changes. WeatherSense on the other hand records information on particular locations and thus the data is more relevant and useful to local users.

Another concept supported by this system is that of smart cities and digital transformation through the introduction of environmental monitoring based on IoT. It promotes the application of contemporary technologies to address the real-life issues, which can enhance the infrastructure planning, emergency response mechanisms, and community understanding of the weather conditions.

The system contributes to continuous monitoring that does not interfere with natural ecosystems in terms of environmental impact. Small low-powered IoT devices make them have a minimum impact on the environment as opposed to conventional weather stations that consumes a lot of infrastructure and power. This renders the system eco-friendly and sustainable.

Additionally, WeatherSense has the potential of aiding disaster preparedness indirectly through the provision of real time reports on the intensity of rainfall, changes in temperature and pressure variations in the atmosphere. Such lessons can be used to guide communities, particularly those in flood prone areas or climatic sensitive areas, to ensure timely preventive actions, which will minimize the possible destruction and loss.

5.2.3 Ethical Aspects

The WeatherSense system has been designed with very high ethical values in order to

make sure that the technology is used responsibly, transparently and safely. Protection of privacy is only one most fundamental ethical considerations. No personal or user-related information is involved and the system simply gathers environmental information of temperature, humidity, air pressure and rainfall. This is to make sure that there are no issues of privacy when using the system.

Data transparency a another crucial ethical consideration. The system offers real-time environmental measurements that are not manipulated or artificially altered by sensors. This will make sure that users get credible and true information, which is the most crucial when weather information is applied to make decisions in emergency cases.

The system will also be educational and research-oriented, i.e. will not imply any harmful and unethical use of the collected data. It intently concentrates on environmental surveillance and prediction to make sure that the technology is utilized in a responsible and significant manner.

Moreover, ethical engineering habits are upheld through fairness, accuracy and reliability of data representation. Although the system might have minor sensor errors related to environmental conditions, the system does not distort or overstate results. Rather, it uses a transparent way of presenting data allowing users to perceive the information appropriately.

In general, the ethical design guarantees reliability, security, and fair utilization of the IoT and machine learning technologies.

5.2.4 Sustainability Plan

The WeatherSense sustainability plan will be aimed at providing the sustainability of operation, scalability, and eco-sufficiency. The hardware components used to construct the system are low-cost and energy-efficient, which include the ESP32 microcontroller and digital sensors. The modules can be operated continuously and with low power usage hence the system is a good fit to be deployed in the long run on real-time.

The system has a modular architecture which is only one of the strengths. All the parts of the system are designed separately, including IoT hardware, backend server, database system, web dashboard and machine learning module, and integrate to form a complete system. Such a modular design contributes to high flexibility of the system so that it can be easily changed or modified in the future without altering the whole system.

As an illustration, more sensor nodes can be installed in other geographical areas to increase the system coverage. Likewise, it is possible to add new capabilities like mobile apps, sophisticated analytics, or processing on the cloud without significant redesign.

Software-wise, scalable technologies such as REST APIs, JSON, and Python frameworks are used, which is why the system can be easily upgraded or moved to cloud platforms later. This helps in long-term flexibility with the changing technology.

WeatherSense is more eco-friendly than large-scale traditional weather stations, which are large infrastructure, maintenance and power intensive. Rather, it relies on distributed IoT devices which are light, efficient and cost-effective.

The system can also be made energy independent and more environmentally friendly in future improvement by powering it with renewable sources of energy like the solar panels. Further, a cloud computing integration and edge AI can further enhance performance, decrease latency, and increase prediction accuracy.

5.3 Project Management and Financial Analysis

WeatherSense project is planned to be a low-cost IoT and web-based application, therefore the primary emphasis of financial planning is to reduce costs of hardware and software development and ensure a reliable operation. Given this is an academic and prototype level system, the budget is maintained at a reasonable level. The financial analysis covers cost of the project (estimated), alternative budget and a basic revenue model in case of future expansion.

Project Cost Analysis (Budget Estimation)

Table 5.1: Proposed System Budget (Final Choice)

Product	Quantity	Cost (BDT) estimated.
ESP32 Microcontroller	1	550 (BDT)
DHT11/DHT22 Sensor	1	150 (BDT)
BMP280 Pressure Sensor	1	300 (BDT)
Rain Sensor Module	1	180 (BDT)
Jumper Wires & Connectors	--	140 (BDT)
USB Cable / Power Supply	1	200 (BDT)
Miscellaneous (resistors, setup)	--	--
Total Estimated Cost	--	1520 – 1600 (BDT)

Rationale:

- This is a cost-efficient and sufficient setting to serve as a working prototype.
- Each component is in favor of real-time IoT data collection.
- Applicable in the final year project of students.

Table 5.2: Alternative Budget (Greater Accuracy System)

Name	Units sold	Approximate price (BDT)
Raspberry PI	1	4500 (BDT)
DHT22 (Higher accuracy sensor)	1	400 (BDT)
BMP280/BME280 Advanced Sensor	1	630 (BDT)
Rain Sensor (Advanced)	1	340 (BDT)
SD Card Module (Data logging)	1	540 (BDT)
Solar Power Module (Optional)	1	700 (BDT)
Additional hardware setup	-	1000 (BDT)
Total Estimated Cost	-	8110 – 9000 (BDT)

Rationale:

- The version is more accurate and durable.
- Fits real world applications and long life.
- Has energy-saving and data storage features.

Cost Comparison Discussion:

- The fundamental model concentrates on low costs and educational application.
- The new model is advanced and is concerned with accuracy, durability, and scalability to the real world.
- In this project, the basic budget is chosen since it offers a good balance in terms of price and functionality.

Future Scope Revenue Model (Future Scope):

Though WeatherSense is principally a research project, it could subsequently be developed into a commercial system in future. Potential revenue models are:

- Farmer or organization-based weather dashboard (paid by subscription).
- Real-time local weather information is available as paid API.
- Agricultural or environmental agency data analytics service.
- Rural and urban Smart IoT weather station sales.
- Connection with mobile application with advanced forecasting capability.

5.4 Complex Engineering Problem

5.4.1 Complex Problem Solving

The system of WeatherSense is a complicated engineering project since it combines IoT devices, and machine learning to monitor the weather and predict it in real-time. The system involves working with several technical areas at a time, such as embedded systems, networking, data processing, and prediction modeling.

Table 5.3: Mapping with Complex Engineering Problem.

EP1 Dept of Knowledge	EP2 Range Of Conflicting Requirements	EP3 Depth of Analysis	EP4 Familiarity of Issues	EP5 Extent of Applicable Codes	EP6 Extent Of Stake-holder Involvement	EP7 Interdependence
✓			✓			✓

EP1- Depth of Knowledge (Justification): The WeatherSense project needs to involve a heavy concentration of the various fields of knowledge such as IoT systems, embedded programming, cloud communication, and machine learning. It entails the implementation of the concepts of both hardware and software engineering to gather and process real-time weather information. System design and implementation requires knowledge of sensor technologies and networking protocols. Thus, the project reflects a profound use of engineering knowledge in various areas.

EP4- Familiarity of Issues (Justification): The system responds to real-life issues like poor local weather forecasting and absence of real-time monitoring of the environment in rural and urban communities. It deals with challenges like sensor noise, data inconsistency, and network reliability in IoT communication. Other problems are also addressed in the project concerning the preprocessing of data and the accuracy of prediction in machine learning models. It therefore needs great exposure to hands-on engineering and environmental issues.

EP7 – Interdependence (Justification): WeatherSense is an extremely interdependent system, where hardware, software, database and machine learning modules are continually dependent on each other. The sensor data received by, ESP32 is directly

related to backend processing, storage, and prediction outcomes. Any sluggishness or fault in one part will affect the overall system performance and dashboard display. Thus, effective operation relies on the good coordination of all the system modules.

Mapping with Knowledge Profile

Table 5.4: Mapping with knowledge Profile.

K1 Natural Science	K2 Mathematics	K3 Engineering Fundamentals	K4 Specialist Knowledge	K5 Engineering Design	K6 Engineering Practice	K7 Comprehension	K8 Research Literature
✓	✓	✓	✓	✓	✓		✓

K1 – Natural Science (Justification): WeatherSense is founded on the natural science ideas which are connected with the weather conditions and the environmental behaviour. It involves making use of real-life physical characteristics such as temperature, humidity, pressure, and rainfall. Proper weathermonitoring and interpretation cannot be achieved without knowledge about these natural phenomena.

K2 – Mathematics (Justification): Mathematics is used to analyze and process sensor data collected from the environment. Statistical and numerical techniques aid in determining trends and predicting the weather patterns. It is necessary in enhancing the precision of the prediction model.

K3 – Engineering Fundamentals (Justification): The project is designed based on fundamental engineering concepts including embedded systems and integration of sensors. The most important elements of the system are ESP32 microcontroller and communication protocols. These basics guarantee the appropriate hardware-softwareinterface.

K4 – Specialist Knowledge (Justification): WeatherSense must have a specialization in IoT systems and environmental sensing technology. It also implies machine learning methods of weather prediction on a short term basis. This area of knowledge is critical in the development of the system.

K5 – Engineering Design (Justification): The system is structured to be an entire IoT architecture that links sensors, server, and dashboard. It guarantees a clear passage of data between hardware and software interface. Good design is required for scalability and real-time performance.

K6 – Engineering Practice (Justification): Practical work will involve hardware configuration, coding, testing and debugging of the system. Calibration of sensors and real-time transmission of data are put into practice. These tests make sure that the

system is functioning under real-world conditions.

K8 – Research Literature (Justification): The project is designed on the basis of studying available IEEE publications and IoT research papers. The use of literature assists in choosing the correct models and enhancing the accuracy of prediction. It makes sure that the system is oriented towards the current researchstandards.

5.4.2 Engineering Activities

WeatherSense is an engineering undertaking that would entail several engineering tasks, which are primarily system integration and real-time data processing.

Mapping with Complex Engineering Activities

Table 5.5: Mapping with Complex Engineering Activities.

EA1 Range of re-sources	EA2 Level of Interaction	EA3 Innovation	EA4 Consequences for society and environment	EA5 Familiarity
✓			✓	✓

EA1 - Range of Resources (Justification): WeatherSense will need several engineering resources such as hardware sensors, microcontroller, cloud/database and web technologies. It is also compatible with software applications such as Flask, SQLite, and machine learning libraries. All these resources are necessary to create a full-fledged system based on IoT.

EA4 - Consequences for Society and Environment (Justification): The system is giving back to the society as it offers real-time weather information that can be used to ensure safety and planning. It assists in the minimization of risks as a result of sudden weather changes including heavy rain or storms. The proper forecasting can help on the side of agriculture and environmental consciousness.

EA5 – Familiarity (Justification): The project is created with the help of familiar technologies, including ESP32, sensors, and web-based dashboards. They are widely applied in the undergraduate level in IoT and embedded system projects. This familiarity helps in efficient development and implementation of the system.

5.5 Summary

This area is a systematic description of the complexity of the WeatherSense system as an engineer. The project will not focus on a specific area of technology but will instead be a synthesis of several areas including IoT-based hardware design, embedded

programming, web development, database management, and machine learning-based prediction. This combination transforms the system into a full-fledged real-time weather monitoring and prediction system where one aspect relies on the right operations of other aspects.

It has been analyzed that the system should deal with several types of engineering issues at once, such as real-time collection of data on sensors, reliable wireless communication with ESP32, effective transfer of data via web APIs, and proper processing of environmental data to make predictions. Moreover, the system should balance various constraints including low cost, energy efficiency, accuracy, and scalability which also further enhance the complexity of engineering the system.

The complex engineering problem categories and knowledge profiles mapping illustrates that WeatherSense must have a broad scope of engineering knowledge. It entails basic science to learn the weather parameters, mathematical concepts to analyze the data and the specialized engineering knowledge to design and implement the systems. In addition, useful engineering knowledge is needed to combine all the parts into a functional system, to provide a seamless communication between hardware and software layers.

In general, this section supports the idea that WeatherSense is a multi-disciplinary engineering initiative, beyond a mere application. It is a practical engineering answer to a real-world problem and it is a practical implementation of the theory of environmental monitoring to address the issues of environmental monitoring efficiently. The system puts emphasis on the role of integration, optimization, and design thinking in designing contemporary IoT-based intelligent systems.

Chapter 6

Conclusion

The WeatherSense project is closed in this chapter with a summary of the systematic development of the system and the results. The system is able to combine IoT and machine learning to deliver real-time weather monitoring, data analysis, and short-term prediction via a web-based dashboard. On the whole, the project has shown a good and viable solution to smart weather tracking and forecasting.

6.1 Summary

The WeatherSense project is an IoT and machine learning-driven smart weather monitor and prediction system created in order to gather, analyze, and present real-time environmental information in an efficient and easy-to-use manner. The system combines several engineering fields such as embedded systems, networking, cloud-based data communication and machine learning to offer a holistic solution to local weather monitoring.

The IoT sensors in this system constantly measure environmental parameters, including temperature, humidity, air pressure, and rainfall and register them with a programmed ESP32 microcontroller. The data collected is sent to a backend server via the use of HTTP-based communication, and processed to be stored in a structured format. The real-time data are visualized in a web-based dashboard whereby users can easily monitor the weather conditions anywhere.

The simplest machine learning algorithm is also applied to examine past trends and give short-term weather forecasts. This introduces a smart dimension in the system and therefore more practical than the traditional non-dynamism systems of monitoring.

General system accomplishments:

- IoT sensors to collect real-time environmental data.
- Wireless data sending with ESP32 and HTTP protocol.
- Live visualization and monitoring Web dashboard.
- Simple machine learning prediction.
- Scalable, low-cost, and modular system design.

6.2 Limitation

The WeatherSense system has a number of limitations that limit it to full scale implementation of a smart weather monitoring system despite it achieving a working prototype of a smart weather monitoring system. These constraints are largely brought about by hardware constraints, data dependency as well as system scalability factors.

One of the significant shortcomings is connected with the sensor accuracy and environmental responsiveness. There is a possibility of low-cost sensors giving slightly uneven readings as a result of calibration drift or environmental interference, or the long-term effects of use. This may lead to inaccuracy of gathered weather data under some circumstances.

The other significant constraint is a small dataset on machine learning prediction. The system is based on relatively limited historical data and, therefore, the model of prediction would be more appropriate to short-term forecasting rather than long-term meteorological predictions, which are highly accurate. To enhance model reliability, more data gathering over a period would be needed.

Real-time in communication between IoT devices also the backend server also requires the system to had consistent internet connectivity. There can be delays in data transmission or temporary disconnection problems that can be experienced in rural or unstable network coverage.

Also, the existing system is programmed to be used on small-scale, i.e. it can be used on a small number of sensor nodes. Architectural enhancements like optimization of cloud infrastructure and load balancing would need scaling it to a large distributed network.

Key_limitations summarized:

- Hardware can be cheap and cause slight inaccuracies in sensor readings.
- Small dataset size is a limitation in machine learning model.
- Needs to be constantly connected to the internet to work in real-time.
- Not designed to run on a large scale (many nodes).
- Power dependency to keep the IoT running.
- Little sophisticated forecasting (only short-term)

Although constrained by these, the system is an effective prototype of a working system and offers a solid base to continue improvement in the future.

6.3 Future Work

WeatherSense system has high prospects of growth and development in the future not only in technical terms, but also in practical application. As it will continue to develop, it can become a fully scalable smart environmental monitoring and forecasting platform.

The introduction of advanced machine learning and deep learning models is one of the most significant future enhancements. With larger datasets and more complex algorithms, the system will be able to be more accurate in both short and long term weather prediction. Such methods as LSTM (Long Short-Term Memory) networks or time-series forecasting models may be presented.

The other significant improvement is the creation of a multi-node system of an IoT network in which numerous weather stations are installed in various geographical locations. This would enable weather analysis in the region and enhance prediction accuracy through the gathering of various environmental data.

The system may be also extended to a mobile application that will inform the user with real-time notification, weather alerts and provide user easy access to the data even without the aid of web browser. This would greatly enhance usability and accessibility.

Regarding infrastructure, cloud computing and edge computing integration can enhance the scalability of the infrastructure, the speed of data processing, and reliability. Long-term data analysis and model of weather trends in the past could also be made possible through cloud storage.

To enhance sustainability, the system may adopt renewable energy sources like the use of solar powered IoT devices particularly in remote locations where there is limited access to electricity. This would render the system more autonomous and eco-friendly.

Moreover, it can be integrated with outside weather APIs and government sources of meteorological data to confirm sensor data and enhance the net accuracy of predictions.

Future enhancement points:

- ML/DL models (LSTM, time-series forecasting) are used.
- Growth to big-scale multi-location IoT network.
- Creation of alert system mobile app.
- Scalability and storage architecture on clouds.
- Solar-powered IoT devices integration to be sustainable.
- External weather API with hybrid to improve accuracy.
- Live warning system on severe weather conditions.

References

[1] J. J. A.-M. Sayor, N. T. Shishir, B. B. Barmon, S. Ahemed, and Md. M. Rahman, "IoT-driven real-time weather measurement and forecasting mobile application with machine learning integration," Array, vol. 27, p. 100474, Aug. 2025, doi: https://doi.org/10.1016/j.array.2025.100474.

[2] T. P. Fowdur and R. M. Nassir-Ud-Diin Ibn Nazir, "A real-time collaborative machine learning based weather forecasting system with multiple predictor locations," Array, vol. 14, p. 100153, Jul. 2022, doi: https://doi.org/10.1016/j.array.2022.100153.

[3] "The Impact of IoT and Sensor Integration on Real-Time Weather Monitoring Systems: A Systematic Review," www.researchsquare.com, Nov. 10, 2023. https://www.researchsquare.com/article/rs-3579172/v1

[4] S. K. Das and P. Nayak, "Integration of IoT- AI powered local weather forecasting: A Game-Changer for Agriculture," arXiv.org, 2024. https://arxiv.org/abs/2501.14754

[5] D. Bhattacharyya and I. Banerjee, "Synergizing IoT and Machine Learning in Hybrid Models for Next-Generation Weather Forecasting," Journal of Emerging Trends in Computer Science and Applications, vol. 1, no. 1, pp. 55–66, Nov. 2025, doi: https://doi.org/10.65525/jetcsa.v1i1.4.

[6] M. Kumari, A. Kumar, S. Singh, and S. Singh, "Multidisciplinary Real-Time Model for Smart Agriculture based on Weather Forecasting Using IoT, Machine Learning, Big Data and Cloud," Oct. 2021, doi: https://doi.org/10.1109/iscon52037.2021.9702455.

[7] T. P. Fowdur, Y. Beeharry, V. Hurbungs, V. Bassoo, V. Ramnarain-Seetohul, and E. C. M. Lun, "Performance analysis and implementation of an adaptive real-time weather forecasting system," Internet of Things, vol. 3–4, pp. 12–33, Oct. 2018, doi: https://doi.org/10.1016/j.iot.2018.09.002.

[8] Abdullah Albuali, Ramasamy Srinivasagan, A. Aljughaiman, and F. Alderazi, "Scalable Lightweight IoT-Based Smart Weather Measurement System," Sensors, vol. 23, no. 12, pp. 5569–5569, Jun. 2023, doi: https://doi.org/10.3390/s23125569.

[9] Bella, M. Khan, M. Naidu, Digumarti Sai Jayanth, and Yousaf Ali Khan, "Developing a Sustainable IoT-based Smart Weather Station for Real Time Weather Monitoring and Forecasting," E3S web of conferences, vol. 430, pp. 01092–01092, Jan. 2023, doi: https://doi.org/10.1051/e3sconf/202343001092.

[10] M. D. S. Sharafat et al., "An IoT-Enabled AI System for Real-Time Crop Prediction Using Soil and Weather Data in Precision Agriculture," Smart Agricultural Technology, p. 101263, Jul. 2025, doi: https://doi.org/10.1016/j.atech.2025.101263.

[11] P. Sharma and S. Prakash, "Real Time Weather Monitoring System Using Iot," ITM Web of Conferences, vol. 40, p. 01006, 2021, doi: https://doi.org/10.1051/itmconf/20214001006.

[12] P. Karvelis, D. Mazzei, M. Biviano, and C. Stylios, "PortWeather: A Lightweight Onboard Solution for Real-Time Weather Prediction," Sensors, vol. 20, no. 11, p. 3181, Jun. 2020, doi: https://doi.org/10.3390/s20113181.

[13] Şükrü Mustafa Kaya, Buket İşler, A. M. Abu-Mahfouz, J. Rasheed, and Abdulaziz AlShammari, "An Intelligent Anomaly Detection Approach for Accurate and Reliable Weather Forecasting at IoT Edges: A Case Study," Sensors, vol. 23, no. 5, pp. 2426–2426, Feb. 2023, doi: https://doi.org/10.3390/s23052426.

[14] P. Varalakshmi, R. C, R. S, R. raj RL, and Rk. R. M, "SkyCast: Intelligent Web-Based Weather Forecasting Using Real-Time Data Analytics," 2026 Second International Conference on Intelligent Systems for Communication, IoT and Security (ICISCoIS), pp. 1–5, Jan. 2026, doi: https://doi.org/10.1109/iciscois62701.2026.11447519.

[15] Avines Panneer Selvam and N. Saud, "Environmental impact evaluation using smart real-time weather monitoring systems: a systematic review," Innovative Infrastructure Solutions, vol. 10, no. 1, Dec. 2024, doi: https://doi.org/10.1007/s41062-024-01817-7.

[16] M. P. Raju and A. J. Laxmi, "IOT based Online Load Forecasting using Machine Learning Algorithms," Procedia Computer Science, vol. 171, pp. 551–560, 2020, doi: https://doi.org/10.1016/j.procs.2020.04.059.

[17] B. İşler, Ş. M. Kaya, and F. R. Kılıç, "Fog-Enabled Machine Learning Approaches for Weather Prediction in IoT Systems: A Case Study," Sensors, vol. 25, no. 13, p. 4070, Jun. 2025, doi: https://doi.org/10.3390/s25134070.

[18] M. Sakib, H. Mia, M. Akter, J. Farduse, S. Akter, and Md. Shahjalal, "Rainfall Prediction and Real-Time Weather Monitoring in Bangladesh: A Comparative Analysis of Machine Learning Algorithms," 2024 21st International Conference on Electrical Engineering, Computing Science and Automatic Control (CCE), pp. 1–6, Oct. 2024, doi: https://doi.org/10.1109/cce62852.2024.10771063.

[19] M. A. Khan, R. Khan, and M. A. Ansari, Application of Machine Learning in Agriculture. Elsevier, 2022.

[20] R. KUMAR, H. SRIVASTAVA, A. YADAV, and A. GUPTA, "Smart Farming & Weather Forecasting with Machine Learning," JOURNAL OF ADVANCE AND FUTURE RESEARCH, vol. 3, no. 11, pp. 607–613607–613, Nov. 2025, Accessed: May 03, 2026. [Online]. Available: https://rjwave.org/jaafr/viewpaperforall.php?paper=JAAFR2511296

23% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

59 AI-generated only 23%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.



False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

221-15-5679

ORIGINALITY REPORT

16%	13%	7%	9%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	dspace.daffodilvarsity.edu.bd:8080 Internet Source	5%
2	Submitted to Daffodil International University Student Paper	4%
3	Silvia Ganesan, Chong Peng Lean, Li Chen, Kong Feng Yuan, Ng Poh Kiat, Mohammed Reyasudin Basir Khan. "IoT-enabled Smart Weather Stations: Innovations, Challenges, and Future Directions", Malaysian Journal of Science and Advanced Technology, 2024 Publication	<1%
4	Submitted to Khwaja Yunus Ali University Student Paper	<1%
5	Submitted to Universitas Diponegoro Student Paper	<1%
6	ouci.dntb.gov.ua Internet Source	<1%
7	Submitted to Polytechnic Institute Australia Student Paper	<1%
8	Submitted to Fiji National University Student Paper	<1%
9	Hemlata Sharma, Shilpa Srivastava, Varuna Gupta. "Application and challenges of optimization in Internet of Things (IoT)", Institution of Engineering and Technology (IET), 2023 Publication	<1%